

An Evaluation of Multi-Agent Reinforcement Learning for Dynamic Bus Scheduling Under Non-Ideal Conditions

A thesis submitted in partial fulfilment
of the requirements for the degree of
Bachelor of Science in Electronics Engineering

Department of Electronics Engineering
Faculty of Engineering
University of Santo Tomas

Proponents:

Badal, Khalil T.
Lopez, Elijah David B.
Mananguit, Jared Asher N.
Marquez, Rence Joseph L.
Medenilla, Jose Anton M.

Research Supervisor:

Asst. Prof. Kanny Krizzy D. Serrano,
MSc

Date: July 6, 2026

Word Count: 13192

AI Use Disclosure

The authors acknowledge the use of artificial intelligence (AI)-assisted tools during the preparation of this manuscript. ChatGPT and Grammarly were used primarily as editorial, organizational, and formatting support tools during the writing and manuscript-preparation process.

Grammarly was used throughout all chapters for grammar checking, spelling correction, punctuation review, and sentence-level clarity suggestions. ChatGPT was used selectively for proofreading and language refinement of draft prose to improve readability, reduce informal phrasing, and smooth transitions between sections. It was also used to provide organizational feedback on manuscript structure, particularly in refining the funnel progression of the Review of Related Work, summarizing disturbance coverage across studies already collected by the authors, checking consistency of terminology, notation, section numbering, and symbol usage across Chapters 1–3, and identifying undefined terms, missing bibliography entries, and citation inconsistencies prior to compilation.

In addition, AI-assisted tools were used for document-preparation and Latex-related support tasks, including table formatting, float placement adjustments, bibliography troubleshooting, citation-key checking, and resolving compilation issues encountered during Overleaf manuscript assembly.

AI tools were employed strictly as editorial, organizational, and formatting assistants during the preparation of this manuscript. They were not used to generate research findings, conduct simulations, or produce technical conclusions. All core research decisions, including the proposed simulation framework, methodological direction, disturbance-modeling approach, evaluation design, and interpretation of the reviewed literature, were determined and validated by the authors. All AI-generated suggestions were reviewed, revised where necessary, and approved prior to inclusion. The authors assume full responsibility for the accuracy, originality, and integrity of the proposed work.

Contents

1	Introduction and Literature Review	6
1.1	Background of the Study	6
1.2	Review of Related Work	9
1.2.1	From Static Rules to Machine Learning	9
1.2.2	Single-Agent Reinforcement Learning and Its Limitations	10
1.2.3	Multi-Agent Reinforcement Learning	11
1.2.4	MARL Applied to Bus Scheduling	13
1.2.5	The Disturbance Gap	15
1.2.6	Significance of Closing the Gap	16
2	Problem Statement	17
2.1	Rationale	17
2.2	Research Gap	18
2.3	Research Objectives	19
2.3.1	Main Objective	19
2.3.2	Specific Objectives	19
2.4	Significance of the Study	19
2.5	Scope and Limitations	20
3	Methods and Research Design	22
3.1	Research Design	22
3.2	Methodology	23
3.2.1	Simulation Environment	23
3.2.2	Control-Stop Selection	24
3.2.3	Environment Model Validation	25
3.2.4	Operating Conditions	26
3.2.5	Data Processing	27
3.2.6	Stochastic Disturbance Generators	28
3.2.7	MARL Architecture and MDP Formulation	32
3.2.8	Baseline Controllers for Comparison	38
3.2.9	Data Analysis Methods	39
3.2.10	Evaluation Methods	40
3.3	Methodological Challenges	41

List of Figures

1.1	EDSA Carousel ridership	7
1.2	Rainfall impact on traffic flow	7
1.3	SARL vs MARL architecture	12
1.4	Centralized training with decentralized execution	13
3.1	Two-phase simulation pipeline	24
3.2	Illustrative SUMO calibration output	25
3.3	Agent–environment cycle (training)	37
3.4	Illustrative Stage A output format	40
3.5	Monte Carlo evaluation layer	41

List of Tables

1.1	Progression from static rules to reinforcement learning along the dimensions that determine real-time control capability.	10
1.2	Disturbance coverage across MARL bus-scheduling literature.	14
3.1	Notation used in the stochastic generators and the MARL formulation.	29

Chapter 1

Introduction and Literature Review

1.1 Background of the Study

The public transportation system has long been a commodity to people; it operates on different terrains, has different forms, and has predetermined routes depending on their geographical location and the needs of commuters [1]. With an increase in urbanization over time, the demand for transportation has also increased. Despite the predicted decline in urbanization rates in Southeast Asia, the Philippines is experiencing the opposite [2]. Due to the increased urbanization and the continuous increase in population (especially in urban areas), people start looking for different alternatives in transportation [3]. With the increasing number of motor vehicles on the road, the traffic congestion worsens, which leads to commuters finding other means to quickly reach their destination. In Metro Manila, severe traffic congestion, stemming from insufficient road infrastructure and poor traffic mitigation policies, has resulted in a highly inefficient public transport system [4]. As a response, the Philippine government launched the EDSA Bus Carousel in 2020. It is the only busway system in Metro Manila implemented by the government to have a designated lane, and separated from other cars by concrete barriers [5]. Throughout the years of implementation, a large number of commuters have utilized the system with an average of 183,000 daily passengers, climbing up to 320,000 during peak periods, with a ridership of nearly 67 million in 2025, up from 63.02 million in 2024 [6]. With that amount of ridership, the EDSA Bus Carousel can be safely assumed to be one of the backbones of Metro Manila commuting.

Despite the gradual increase in demand, the EDSA Bus Carousel still faces a lot of significant operational issues that affect the quality of bus routing. Firstly, the LTFRB relied on a tedious manual monitoring process by deploying personnel at each stop, creating decisions through empirical observation. [7]. In relation to that, it is also clear that bus bunching has not been resolved; most of the time, their designated dispatch schedule is not followed [8]. There is an acknowledged lack of a structured monitoring framework for operations (e.g., dispatching, dwell time, bus bunching), in which adjustments are often done only after recurring issues surface [9]. Moreover, mechanical failures or bus breakdowns are also prominent in Metro Manila, which contributes to a stressful environment, highlighting that poor

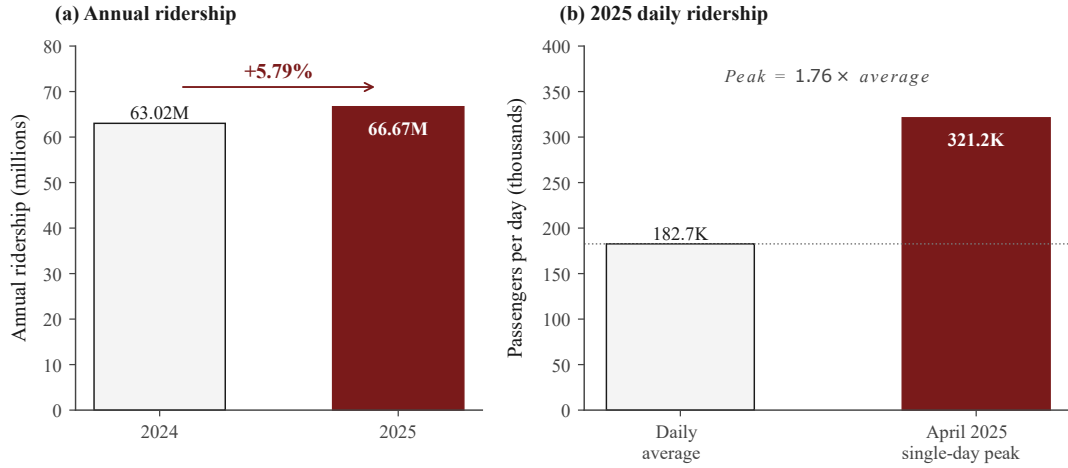


Figure 1.1: EDSA Carousel ridership. (a) Annual ridership grew 5.79% from 63.02M in 2024 to 66.67M in 2025. (b) Daily ridership in 2025 averaged 182,655 passengers, with a single-day peak of 321,186 in April 2025 — approximately $1.76\times$ the daily mean [6].

vehicle condition and the constant threat of mid-route breakdowns are significant chronic stressors for Filipino bus drivers [5]. Furthermore, environmental stochasticity severely destabilizes operations: empirical studies on Philippine expressways show that increasing rainfall intensity significantly reduces average traffic speed and free-flow capacity [10].

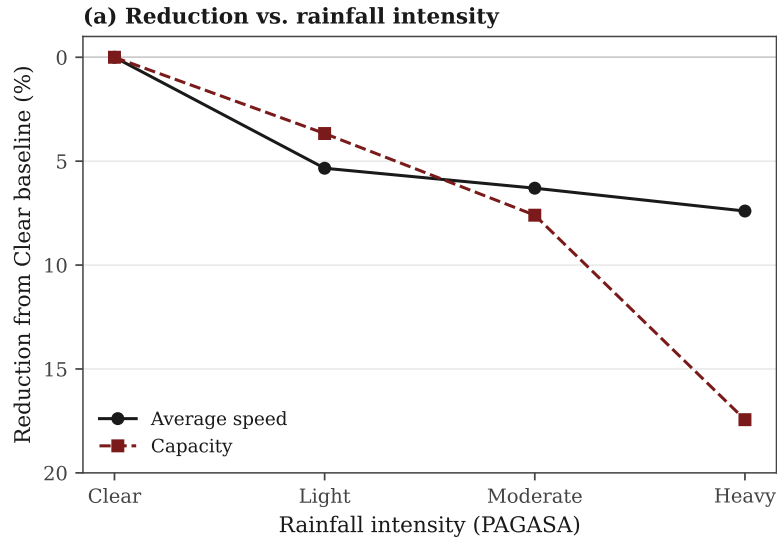


Figure 1.2: Reduction in average speed and free-flow capacity relative to the clear-weather baseline across PAGASA rainfall categories. Capacity loss grows sharply at heavy intensities (-17.4%), whereas mean speed degrades more moderately (-7.4%), adapted from [10].

The reduction in average speeds are about 5.34% under light rain conditions, 6.3% under moderate rain conditions, and 7.4% under heavy rain conditions. The average volume decreased by 2.92%, 10.62%, and 12.39% while the capacities reduced by 3.67%, 7.6%, and 17.44% under light, medium, and heavy rain conditions, respectively [10]. Severe weather events often initiate localized flooding at key seg-

ments like Muñoz and Monumento Stations or cause severe degradation, forcing operational halts and emergency speed restrictions that further affect the busway’s performance [11–13]. Given these operational hurdles, the bus dispatching process of the EDSA Carousel can be modeled as a Vehicle Scheduling Problem (VSP).

Traditionally, the VSP has been addressed through static scheduling, which utilizes fixed timetables and headways derived from historical averages [14]. However, these rigid, rule-based methods often fail in stochastic environments as they cannot adapt to real-time disruptions, necessitating a shift toward more dynamic, autonomous scheduling frameworks that can respond to the unpredictability of the corridor [8]. This motivated a transition toward data-driven methodologies. Initial studies utilized traditional machine learning (ML) for demand forecasting and traffic prediction [15], but these models are primarily limited to predictive tasks and lack the active control mechanisms required for real-time decision-making in a closed-loop system. The subsequent wave introduced Single-Agent Reinforcement Learning (SARL), which enabled autonomous control, yet these frameworks frequently suffer from the ‘curse of dimensionality’ as the joint state and action spaces grow exponentially with the number of vehicles, leading to centralized bottlenecks that hinder scalability in large transit networks [16]. To overcome these limitations, Multi-Agent Reinforcement Learning (MARL) has emerged as the state-of-the-art approach, offering a decentralized execution framework that enables scalable, collaborative control among multiple buses to mitigate complex issues like bus bunching [17].

Although recent studies have demonstrated the potential of MARL-based dynamic scheduling to improve transit adaptability and overall reliability, a critical research gap remains. Existing MARL bus scheduling models are primarily evaluated in controlled simulation settings and are typically tested against isolated disruptions. No prior work has evaluated these models under the combined, simultaneous severe disruptions such as heavy rain, mechanical breakdowns, traffic variations, and sudden passenger surges that portray real-world corridors like EDSA. Therefore, this study aims to evaluate a MARL-based dynamic bus scheduling model under these non-ideal, stochastic traffic conditions to mitigate bus bunching, minimize passenger waiting times, and improve overall dispatch efficiency on the EDSA Carousel.

1.2 Review of Related Work

This section surveys prior work on data-driven bus and traffic scheduling using a funnel structure. It starts from the broad transition from static scheduling toward adaptive and learning-based control, narrows to single-agent reinforcement learning and its known scalability limitations in large transit systems, narrows further to Multi-Agent Reinforcement Learning (MARL) and the assumptions commonly used in existing MARL bus-scheduling studies, and finally identifies the robustness gap addressed in this study.

1.2.1 From Static Rules to Machine Learning

Static scheduling refers to the offline determination of bus departure times, headways, fleet allocations, and stop assignments based on historical demand and travel-time averages, fixed in advance of operations and not adjusted in real time in response to observed disturbances [14]. Once deployed, such schedules cannot react to live disturbances such as demand surges, congestion, weather-induced delays, or vehicle breakdowns. The poor performance of fixed timetables under such conditions is well documented: even-headway and schedule-based control degrade rapidly once travel-time and demand variability exceed the narrow band assumed at design time, allowing small headway perturbations to amplify into bunching [18]. Static schedules therefore remain mathematically inadequate for stochastic traffic environments, where the governing quantities are random variables rather than deterministic constants. These limitations motivated the transition toward more adaptive and data-driven scheduling methodologies.

Machine Learning (ML) techniques subsequently emerged as important tools for improving operational responsiveness in transportation systems through demand estimation, traffic forecasting, predictive dispatching, and adaptive holding strategies [19, 20]. Unlike static scheduling approaches, ML-assisted frameworks incorporate continuously updated operational information to improve short-term decision support under uncertain traffic conditions [19]. Recent applications to transit operations have demonstrated measurable operational gains. Barrera Hernandez et al. applied passenger-demand forecasting to support a heuristic dispatcher for bus operations in Bogotá, reporting reductions in passenger waiting time and operational cost relative to fixed timetables [21]. Similarly, Usman et al. utilized ML-based density estimation to improve real-time responsiveness in public transit systems [22]. These studies demonstrate the growing role of ML-assisted methodologies in adaptive transit scheduling and operational planning.

Despite these advances, many ML-assisted transit-control frameworks still rely heavily on manually engineered intervention logic and externally designed dispatch heuristics [23]. In several predictive-control and heuristic-dispatch systems, forecasting components improve operational visibility, but the resulting operational responses are still governed by manually specified holding rules, threshold conditions, or predefined intervention strategies [24, 25]. In the framework proposed by Barrera Hernandez et al., for example, the forecasting model informs the dispatcher but does not autonomously determine operational actions such as holding duration, dispatch timing, or stop-skipping behavior [21]. Consequently, while ML-assisted scheduling frameworks improve anticipatory planning and adaptive responsiveness, many ex-

isting implementations still stop short of establishing a fully integrated closed-loop control framework capable of autonomously optimizing sequential dispatch decisions over time.

Table 1.1: Progression from static rules to reinforcement learning along the dimensions that determine real-time control capability.

Paradigm	Adapts to real-time conditions?	Produces a dispatching action?	How the action is determined
Static / heuristic scheduling	No, fixed offline from historical averages	Yes, but actions are pre-determined before operation	Pre-set timetables or manually designed heuristic rules that do not adapt to live corridor conditions [14, 18]
Machine-learning assisted scheduling	Yes, forecasts update continuously using observed operational data	Partially; predictions support decisions, but actions are still governed externally	Forecasting models estimate demand or traffic conditions while separate heuristic dispatch rules determine the operational response [21, 22, 24]
Reinforcement learning	Yes, policies continuously condition on live observed system states	Yes, the dispatching action is produced directly by the learned policy	The control policy learns through interaction with the environment to maximize long-term operational performance [26–28]

The decisive distinction is the final column: only reinforcement learning makes the dispatching action itself a learned output. This study instantiates the reinforcement-learning row as a parameter-sharing Double Deep Q-Network under centralized training with decentralized execution (Chapter 3).

1.2.2 Single-Agent Reinforcement Learning and Its Limitations

Reinforcement Learning (RL) closes the prediction–control gap by learning a *policy*: a mapping from observed system states to actions, optimized through trial-and-error interaction with the environment to maximize a long-horizon reward signal [26]. Unlike a forecaster, an RL agent does not just predict what will happen — it decides what to do. The reward signal closes the loop: the agent’s action changes the environment, the environment returns a new state and a scalar reward, and the policy is updated so that better actions become more likely the next time a similar state is encountered.

When RL is applied to a multi-vehicle problem such as bus-fleet control, the simplest formulation treats the whole fleet as one entity controlled by a single agent. This is **Single-Agent Reinforcement Learning (SARL)**: one centralized policy observes the full system state and emits a joint action covering every bus at once. SARL has been applied directly to bus holding control, including the STDH-DQN approach of Zhao et al. [29], which uses a self-attention state encoder over spatial-temporal AVL features, and the more recent SA-DRL approach of Zhang and Zheng [30], which encodes categorical identifiers (vehicle ID, station ID, trip ID) as state features and reports favorable results against MARL baselines on a bidirectional timetabled line. SARL is therefore a real and competitive option in this space; the question is not whether it works but where it begins to strain.

SARL faces three scalability limitations in multi-bus control, each documented in the bus-scheduling and traffic-control literature. These limitations motivate the move to a multi-agent formulation in the next subsection; each has a corresponding

design response that MARL provides.

1. **State-space dimensionality growth.** A SARL formulation of N buses on M stops requires a global state $s \in \mathbb{R}^{N \cdot d}$, where d is the per-bus feature count (position, headway, load). For the EDSA Carousel sub-corridor studied here, with $M = 24$ stops and a fleet of $N \approx 12\text{--}30$ buses, $\dim(s)$ reaches roughly 96–240. The function approximator must also learn approximate permutation invariance over buses, the policy should not depend on which arbitrary index a bus is assigned, a property not enforced by default in neural networks [31].
2. **Joint action-space combinatorial growth.** A SARL controller emits a joint action $a = (a_1, \dots, a_N) \in A^N$. With the hybrid action space adopted in this study ($|A_i| = 5 \times 2 = 10$, combining a 5-bin holding-strength discretization with a binary skip decision), a SARL controller managing $N = 20$ buses operates over a joint action space of 10^{20} combinations. Although deep RL policies do not enumerate these combinations directly — they sample from a parameterized distribution over actions — the credit-assignment burden remains: a single scalar reward must be attributed across all N simultaneous component actions, and this attribution problem worsens as N grows. MARL with shared policies factorizes the action selection into N independent decisions of size 10, each with its own attributable reward signal.
3. **Sample inefficiency under non-stationarity.** SARL treats the fleet as a single dynamical system; every demand change, breakdown, or weather event triggers re-adaptation of the entire policy. MARL with parameter sharing benefits from *experience aggregation*: every action by every bus generates one training sample for the same shared network, so each bus’s experience accelerates learning for all of them [32, 33].

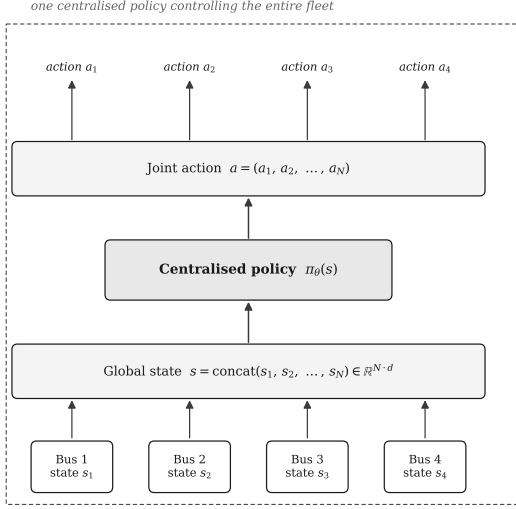
Recent SARL-for-bus-control work that competes successfully against MARL baselines does so partly by hand-engineering categorical agent-identity features into the state representation [30]. This is informative because it suggests that agent identity remains an important representational issue in SARL formulations even at the high end of reported performance — the dimensionality concern in item (1) above is partially addressed by smuggling identity through input features rather than by a change in policy class. The fix is workable on a fixed corridor but does not transfer cleanly when fleet size or route topology changes.

We note a counter-position. Zhang and Zheng [30] report SA-DRL matching or exceeding MARL baselines on bidirectional timetabled lines, attributing this to data-imbalance issues in MARL when some agents see few episodes. The EDSA Carousel sub-corridor studied here is operated as a high-frequency BRT service along a defined directional segment, closer to the single-direction loop-line assumptions where MARL has been shown to excel, which motivates the MARL choice here while acknowledging this caveat.

1.2.3 Multi-Agent Reinforcement Learning

Multi-Agent Reinforcement Learning (MARL) addresses the three limitations above by decomposing decision-making across multiple agents that share the environment. Each limitation has a direct design counterpart in MARL: state-space growth is contained by giving each agent only its own *local* observation rather than the global state; action-space growth is factorized by having each agent select only

(a) **Single-Agent RL (SARL)**



(b) **Multi-Agent RL (MARL)**

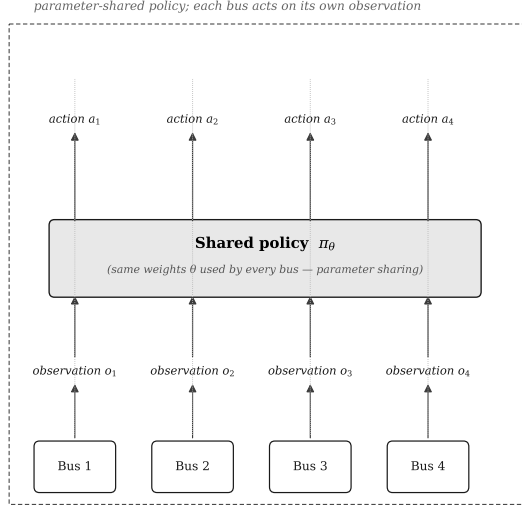


Figure 1.3: Comparison of single-agent and multi-agent formulations. (a) SARL: a centralized policy $\pi_\theta(s)$ consumes the concatenated global state $s \in \mathbb{R}^{N \cdot d}$ and outputs a joint action covering every bus. (b) MARL with parameter sharing: a single set of weights θ is shared across N agents, each acting on its own local observation o_i .

its own action; and the sample-inefficiency problem is mitigated by sharing one network’s parameters across all agents, so that every agent’s experience updates the same policy. The problem is typically formalized as a Markov Game built on an underlying Markov Decision Process, with each agent maintaining its own local observation and acting under a shared or coordinated policy [34–38].

This decomposition introduces a new difficulty: *partial observability*. No individual bus can see the full corridor state; it knows only its own headway, load, and immediate surroundings. A fully decentralized scheme in which each agent both trains and acts on its local observation alone would be unstable, because the other agents’ simultaneously evolving policies make the environment non-stationary from any one agent’s viewpoint — the same local state can transition to different outcomes depending on what the other agents happen to be learning at that moment.

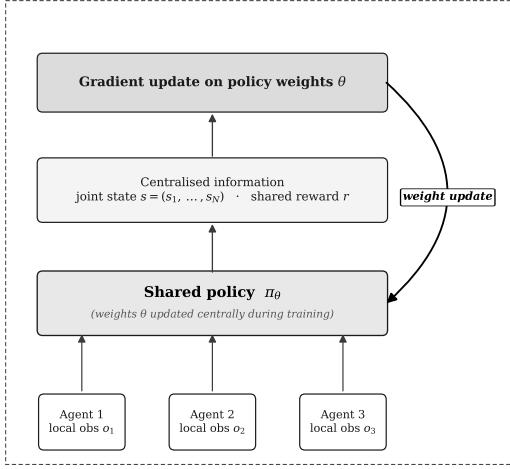
To address this, most modern formulations use **Centralized Training with Decentralized Execution (CTDE)**. During training, agents are allowed to access system-wide information and optimize a shared objective; this stabilizes learning despite the non-stationarity introduced by simultaneously adapting agents. At deployment, each bus acts only on its own local observation, which is what makes the framework practically deployable on a real fleet without requiring a centralized real-time controller [39–42].

In the bus-scheduling setting, a parameter-sharing CTDE architecture is fleet-size invariant: the same trained policy applies regardless of how many buses are currently in service, because every bus is an instance of the same agent class acting on its own local headway, occupancy, and queue observations. Adding or removing a bus does not invalidate the trained model. Christianos et al. [32] demonstrate that parameter sharing scales effectively to large agent populations and provides empirical support for the fleet-size invariance claim.

Recent progress has combined MARL with deep learning algorithms — Deep Q-

(a) Training phase — centralised

agents share one reward signal and one set of policy weights



(b) Execution phase — decentralised

weights frozen; each bus acts on its local observation alone

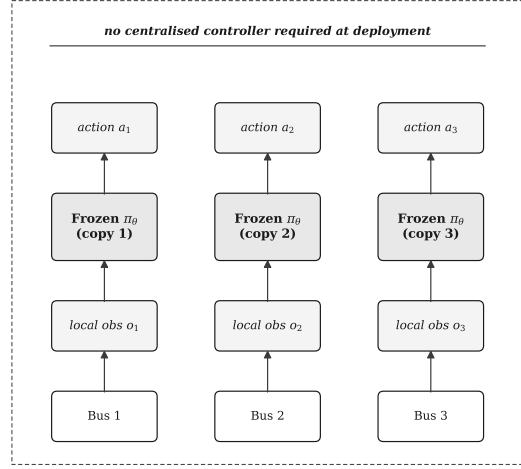


Figure 1.4: (a) During training, all agents share one reward signal and one set of policy weights θ , which are updated centrally using joint-state information. (b) At deployment, the weights are frozen and each bus runs an identical copy of π_θ on its own local observation o_i alone, so no centralized real-time controller is required.

Networks (DQN), Proximal Policy Optimization (PPO) and its multi-agent variant MAPPO, and Graph Attention Networks (GAT) — to handle high-dimensional observation spaces and complex interactions. These have been applied across vehicle scheduling, signal control, and logistics, generally reporting improvements in routing efficiency, congestion, and throughput in simulation [34–40].

1.2.4 MARL Applied to Bus Scheduling

Within bus scheduling specifically, the MARL literature has accumulated along a recognizable trajectory: first establishing that holding control could be learned at all, then enriching the action and objective space, and most recently beginning to examine policy behavior under disturbance. This trajectory is what the present study extends.

The foundational result is due to Wang and Sun [17], who showed that a cooperative MARL framework could learn an effective bus-holding policy on a single-line corridor and outperform classical headway-equalization rules under idealized stochastic demand. Chen et al. [43] provided an earlier proof-of-concept in the same vein on a corridor with multi-agent reinforcement learning over local headway observations. These early results established the viability of the approach but were limited to the holding action alone, restricting the policy’s leverage once bunching had already developed.

Subsequent work enriched the formulation along two axes. Rodriguez et al. [27] extended the action space by combining holding with stop-skipping in a cooperative DDQN framework on a simulated transit corridor with Poisson-distributed passenger arrivals and Gaussian inter-stop travel times, reporting reductions in average passenger waiting time relative to an even-headway holding baseline. Wang and Sun [31] extended the objective space, scaling MARL to multi-line services with a shared corridor and jointly optimizing headway regularity against operational cost

under controlled simulated demand and travel-time profiles.

Recent work has increasingly examined policy behavior under disturbance. Wang and Sun’s IQNC-M [28] introduced implicit quantile networks and meta-learning for risk-sensitive bus holding, evaluating performance under demand surges and traffic-speed perturbations but not under weather-induced heavy-tailed delays or discrete vehicle breakdowns. In adjacent settings, Katzilieris et al. [44] integrated MADDPG-based traffic signal control with dynamic bus lane access management on a simulated arterial network with deterministic signal cycles; Liu et al. [45] reported DRL-based holding control under stochastic travel time and demand; and Shi et al. [46] reported a distributed DRL bus control system in a connected environment with breakdown events modeled but without weather effects.

Across this trajectory, the simulation environments share three idealizing assumptions: passenger arrivals are stationary Poisson processes, inter-stop travel times are symmetric and well-behaved (Gaussian or similar), and the fleet experiences no discrete failure events. Reported performance gains are therefore conditional on those assumptions holding at deployment. The EDSA Carousel sub-corridor studied here violates each of them. Travel times are highly non-stationary, with tail behavior driven by tropical-cyclone and heavy-rainfall events that suspend operations entirely on the most severe days [47]; passenger demand swings sharply across peak intervals, with single-day ridership peaks roughly $1.8\times$ the daily average [6]; and tight headways mean that any individual vehicle breakdown propagates through the corridor as a downstream demand-absorption problem rather than as an isolated event. A controller whose reported gains rest on the idealizing assumptions above has no validated behavior on a corridor that violates all three at once.

Table 1.2: Disturbance coverage across MARL bus-scheduling literature.

Paper	Algorithm	Environment	Disturbances	Reported gain
Wang & Sun (2020) [17]	Cooperative MARL holding	Single-line, idealized	D	Bunching reduction vs. headway rule
Chen et al. (2016) [43]	Early MARL holding	Corridor, idealized	D	Proof-of-concept gains
Rodriguez et al. (2023) [27]	Cooperative DDQN (hybrid action)	Chicago, mesoscopic	T	Wait time vs. headway-control baseline
Wang & Sun (2023) [31]	Multi-objective MARL	Shared corridor, idealized	D	Multi-line bunching reduction
Wang & Sun (2023) [28]	IQNC-M (distributional)	Real-world bus services	S, T	Stable control under extreme events
Katzilieris et al. (2026) [44]	MADDPG	Athens arterial, SUMO	T	Throughput improvement
Liu et al. (2023) [45]	DRL holding	Synthetic single line	D, T	Wait time vs. no-control
Shi et al. (2022) [46]	Distributed DRL	Connected environment	D, B	Integrated control gains
This study	DDQN + PS (CTDE)	Calibrated EDSA SUMO	S, T, W, B	To be evaluated

D = stochastic demand, S = demand surges, T = traffic-speed perturbation, W = weather-induced heavy-tail delay, B = discrete vehicle breakdown.

Table 1.2 summarizes what each study evaluated, what disturbances it modeled, and what it reported. No prior MARL bus paper has evaluated controller performance under a framework that incorporates all four disturbance classes documented in the EDSA corridor’s operational profile such as demand surges, traffic-speed variability, heavy-tailed weather-induced travel-time delays, and discrete breakdowns in a calibrated real-corridor microsimulation. Of these four, weather-induced heavy-tailed delays and discrete breakdowns are the two that no prior MARL bus-scheduling study has modeled at all; their combination with demand surges and a calibrated traffic-speed baseline is the specific disturbance profile introduced here. The algorithmic choice this study adopts to address that combined-disturbance setting, DDQN with parameter sharing under CTDE is justified in detail in Chapter 3 (Section 3.2.7), alongside the rationale for not using actor-critic or distributional alternatives. The action space the agents operate on is correspondingly small and discrete: at each control event, an agent selects a holding strength from a five-bin discretization $\alpha \in \{0.0, 0.1, 0.2, 0.3, 0.4\}$ (multiplied by a maximum holding duration to yield seconds held) combined with a binary decision to skip the next stop, giving $|A_i| = 10$ actions per event. The full action-space rationale, including why discretization is preferred over a continuous holding parameter, is given in Chapter 3 (Section 3.2.7).

Under the non-ideal evaluation regime the weather generator (W) **replaces** the traffic-speed generator (T) as the source of inter-stop travel-time variability rather than acting alongside it. Both are anchored to the same per-segment empirical mean μ , and the weather coefficient of variation ($\eta \geq 0.3$) already spans and exceeds the $\pm 20\%$ range produced by traffic-speed scaling (Section 3.2.6). T therefore governs travel-time stochasticity under the ideal regime and W governs it under the non-ideal regime; the two are not composed on the same segment.

1.2.5 The Disturbance Gap

The idealized assumptions in Table 1.2 limit how confidently the reported MARL gains can be expected to transfer to real urban corridors. Real corridors violate all three assumptions: demand exhibits non-stationary surges, weather-induced travel times are skewed and heavy-tailed, and individual buses break down stochastically. Wang and Sun themselves identify this as central, noting that existing MARL bus-control studies overlook performance under perturbations and that this becomes critical when transferring models toward real-world deployment [28]. Their IQNC-M framework is among the first to address demand surges and traffic-speed perturbations in bus holding control, but weather-induced heavy-tailed delays and discrete vehicle breakdowns remain outside its tested scenario set.

Two considerations explain why the gap has persisted:

1. **Training instability under heavy-tailed returns.** Prior MARL bus-scheduling studies have typically used Poisson arrivals and Gaussian travel-time noise because these distributions generally produce more statistically stable training behavior than heavy-tailed disturbances. In reinforcement learning literature, heavy-tailed returns and stochastic gradients are known to introduce convergence difficulties and unstable optimization behavior during policy learning [48, 49].

2. **Lack of temporally aligned anomaly data.** Historical severe-weather records are rarely aligned with operational bus data at the per-segment granularity needed for direct parameter estimation, making it difficult to fit an empirically grounded heavy-tailed disturbance distribution from corridor-specific data alone, especially given the short operational observation windows typically available. *This study addresses the obstacle by adopting the lognormal travel-time parameterization that Patil et al. [50] validated against INRIX freeway data via the Kolmogorov-Smirnov test, and then sweeping the disturbance intensity across a range of validated and extrapolated CV values rather than fitting a single point estimate. This characterizes controller behavior as a function of disturbance magnitude and avoids reliance on a corridor-specific severe-weather sample whose variance cannot be reliably resolved from a short observation window.*

1.2.6 Significance of Closing the Gap

This study contributes the first characterization of MARL bus-scheduling performance across a swept range of disturbance intensities on a calibrated real-corridor simulation, with the disturbance set covering heavy-tailed weather-induced travel-time delays and discrete bus breakdowns that no prior MARL bus-scheduling study has modeled. Understanding policy performance under disturbance matters for two reasons. First, deploying MARL bus-scheduling models on the basis of ideal-simulator benchmarks alone leaves their real-world failure modes unknown until deployment, which is an unacceptable risk profile for a system that carries hundreds of thousands of daily passengers. Second, the relevant performance question for transit operators is not only average-case waiting time under quiet conditions but how a control policy degrades as a disruption intensifies, a characterization that prior MARL bus-scheduling work does not report. Establishing a reproducible stress-testing protocol pairing a calibrated corridor simulation with configurable demand, weather, and breakdown generators therefore contributes both an empirical characterization of performance under disturbance and a reusable evaluation framework for subsequent MARL work in this domain [34, 37, 38, 40, 51].

Chapter 2

Problem Statement

2.1 Rationale

The public transportation system has long been a commodity that comes in the form of bus transport, train networks, and the like, with predetermined routes and fares [1]. In Metro Manila, traffic congestion has long been a problem stemming from insufficient road infrastructure and poor traffic mitigation policies, resulting in an inefficient public transport system [4]. One of the government's actions for an alternative transportation system that runs within the metropolitan area is the implementation of the EDSA Carousel in 2020. It is the only busway system in Metro Manila implemented by the government to have a designated lane, separated from other vehicles by concrete barriers [5]. Over the years, the number of commuters using the EDSA Carousel has steadily risen, averaging approximately 1.83×10^5 daily passengers, with single-day peaks exceeding 3.2×10^5 during high-demand periods, reaching a new high of nearly 67 million in 2025, up from 63.02 million in 2024 [6].

However, despite its growing demand, the system still faces several problems that can affect the quality of bus routing. To start, the LTFRB relied on a tedious manual monitoring process by deploying personnel at each stop [7]. There are also certain portions where it operates with mixed traffic, causing delays among buses. It is also clear that bus bunching has not been resolved; most of the time, their designated dispatch schedule is not followed [8]. Moreover, the operation of the bus system is also largely influenced by external factors, such as weather and road maintenance. Sections of the EDSA Bus Carousel routes typically close during tropical cyclones to prevent potential damage from continuous rain and strong winds [47]. Given these operational issues, the bus dispatching process of the EDSA Bus Carousel can be modeled as a Vehicle Scheduling Problem (VSP), in which buses must be dispatched and coordinated efficiently despite these issues. By improving the scheduling, operational issues, such as bus bunching, irregular dispatch intervals, and passenger waiting times, will be reduced, thereby improving the overall bus dispatch efficiency.

Busway systems operate in a highly dynamic, unpredictable environment. In the past, Vehicle Scheduling Problems (VSP) relied on traditional scheduling techniques that focused on finding the optimal path by determining the shortest distance. Static scheduling is the pre-computed timetabling that fixes departure times before operation begins, with no mechanism to revise them in response to real-time conditions

[39, 41, 52]. However, because transit environments are inherently stochastic, these static schedules perform poorly during daily operations, which causes severe operational issues such as bus bunching, node failures, and an increase in passenger wait times [18, 53].

To address the rigidity of static schedules, recent scheduling studies have shifted toward dynamic, AI-driven approaches, with Multi-Agent Reinforcement Learning (MARL) as the leading solution. Past studies have successfully utilized MARL to overcome the scalability limits of earlier systems by distributing the scheduling decisions across autonomous agents, improving the overall system’s adaptability to real-time traffic [27, 28, 31]. However, a significant gap remains in how these state-of-the-art models are evaluated. Existing literature primarily validates the MARL scheduling models in a controlled, simulated environment. Previous evaluations rarely account for the heavy-tailed, asymmetric disruptions characteristic of actual transit operations, leaving their performance under non-ideal, real-world traffic anomalies largely untested. To the best of our knowledge, no existing MARL bus-scheduling model has been sufficiently tested against severe, simultaneous disruptions such as sudden passenger surges and weather-induced delays. To bridge this gap, this study develops and evaluates a MARL-based bus scheduling model within a stochastic traffic simulation subjected to non-ideal conditions.

This study develops and evaluates a MARL-based bus scheduling model on a calibrated stochastic simulation of the EDSA Carousel corridor, with non-ideal conditions operationalized as the simultaneous and independent injection of passenger demand surges, severe-weather travel-time skew, and Poisson-distributed bus breakdowns. The objective is to quantify how MARL performance degrades from its ideal-condition baseline as disturbance intensity rises, producing the first integrated robustness characterization for MARL bus scheduling under this specific combination of disturbances.

2.2 Research Gap

Existing Multi-Agent Reinforcement Learning frameworks for bus scheduling demonstrate measurable improvements in passenger waiting time and headway regularity, but these gains are validated almost exclusively in simulation environments that assume stationary passenger arrivals, symmetric Gaussian travel times, and a failure-free fleet. The performance of MARL bus-scheduling models has not been characterized when these idealizing assumptions are simultaneously violated by (i) stochastic passenger demand surges, (ii) heavy-tailed weather-induced travel-time delays, and (iii) discrete bus breakdowns. Without this characterization, it cannot be determined whether reported MARL gains persist, degrade gracefully, or collapse under realistic operating disturbances, which in turn blocks the transition of MARL bus scheduling from simulation to real urban transit deployment.

2.3 Research Objectives

2.3.1 Main Objective

To investigate the performance of a developed Multi-Agent Reinforcement Learning (MARL) scheduling model for dynamic bus scheduling under non-ideal traffic conditions on a calibrated simulation of the EDSA Carousel corridor.

2.3.2 Specific Objectives

1. **Environment Construction:** To construct a stochastic traffic simulation environment representing a defined traffic area.

Expected Output 1.1: A simulation model of a defined operational scope, validated against empirical traffic data to establish an environment that accurately approximates real-world traffic.

Expected Output 1.2: An environment simulator capable of introducing environmental variability through adjustable parameters to simulate non-ideal operating conditions.

2. **Algorithm Implementation:** To develop and implement a Multi-Agent Reinforcement Learning (MARL) approach for dynamic bus scheduling.

Expected Output 2.1: A formulated agent architecture defining individual bus units as agents with discrete state and action space with a continuous reward function that punishes irregularities and passenger service delays.

3. **Performance Evaluation Under Ideal and Non-Ideal Conditions:** To evaluate the scheduling performance of the MARL approach under both ideal operating conditions and non-ideal conditions.

Expected Output 3.1: A quantitative and statistical assessment of MARL scheduling performance under ideal conditions, measuring passenger waiting time, travel time, and headway coefficient of variation, tested against baseline methods.

Expected Output 3.2: A quantitative and statistical assessment of MARL scheduling performance under non-ideal conditions, measuring passenger waiting time, travel time, and headway coefficient of variation, reported as degradation relative to ideal-conditions performance.

2.4 Significance of the Study

This study contributes both practical and scientific significance.

Practical significance. The EDSA Carousel carries on the order of 1.8×10^5 passengers daily, with single-day peaks exceeding 3.2×10^5 [6]. At this scale, even a small per-passenger improvement aggregates into a large operational effect: an illustrative 30-second reduction in mean passenger waiting time, applied across average daily ridership, corresponds to roughly 1.5×10^3 passenger-hours saved per day. (This figure is illustrative of the scale of impact rather than a performance target; actual savings depend on how uniformly the reduction is realized across riders and

operating periods.) Beyond average-case waiting time, this study reports controller performance as a function of disturbance intensity, characterizing how a learned policy degrades as operating conditions worsen — a perspective directly relevant to transit operators making service-reliability decisions during disruptions.

Scientific significance. The primary scientific contribution is empirical: this study produces the first characterization of whether reported MARL bus-scheduling gains persist, degrade gracefully, or collapse when the idealizing assumptions of prior work are violated simultaneously by demand surges, traffic-speed perturbations, weather-induced heavy-tailed delays, and discrete breakdowns. To produce this characterization reproducibly, the study contributes a stress-testing protocol for bus-scheduling reinforcement learning: a calibrated corridor model paired with four configurable stochastic anomaly generators. Because the protocol fixes the corridor, the disturbance generators, and the evaluation metrics, subsequent MARL work can be benchmarked on the same basis, allowing performance claims to be compared rather than reported in isolation. The study further reports the performance-degradation curve of MARL as a function of disturbance intensity, characterizing robustness rather than only average-case performance.

2.5 Scope and Limitations

Scope. This study develops and evaluates a MARL-based bus scheduling framework for the EDSA Carousel corridor. The framework is built on a calibrated SUMO microsimulation and runs over a single-day operational horizon; the simulation horizon, fleet size, and stop set are specified in Chapter 3. Each bus is modeled as an independent agent sharing one learned policy, observing only local conditions and acting at designated control stops. The MARL policy is benchmarked against three baselines — No Control, Forward Headway, and Even Headway — using passenger waiting time, total travel time, and headway coefficient of variation as metrics, over at least 30 Monte Carlo runs per regime. The same trained policy is then evaluated under two regimes, ideal and non-ideal, where non-ideal conditions are defined below.

For this study, **non-ideal conditions** are operating regimes in which one or more of four disturbance classes are active: (i) passenger demand surges, where boarding demand is scaled up above its baseline; (ii) traffic-speed perturbations, where corridor cruising speed varies around its baseline to represent everyday congestion friction; (iii) severe-weather travel-time delays, where inter-stop travel times are drawn from a heavy-tailed distribution to capture rain-induced slowdowns; and (iv) discrete bus breakdowns, where individual vehicles fail at random and leave service. The exact distributions and parameter ranges for each class are specified in Chapter 3. The severe-weather generator characterizes controller behavior across a range of travel-time variability magnitudes rather than across named meteorological categories; mapping specific intensity values to operational weather labels (e.g. “moderate rain” versus “typhoon-grade”) is deferred to follow-on work that pairs the framework with a corridor-specific severe-weather travel-time dataset.

Delimitations. (a) Due to computational constraints, the simulation is restricted to a defined operational sub-segment of the EDSA Carousel corridor rather than the entire metropolitan road network. The restriction is justified by the need to preserve 1:1 empirical traffic volumes for GEH calibration without resorting to flow scaling; corresponding GEH calibration statistics are reported in Chapter 4. (b) MARL control actions are applied only at designated control stops, selected following criteria established in the literature (see Chapter 3). (c) The study is entirely simulation-based; no on-road deployment is performed. (d) Weather and breakdown effects are modeled as statistical distributions of their impact on travel time and fleet availability, not as physical or mechanical simulations of rain dynamics or vehicle subsystems. (e) Only the four disturbance classes defined above are considered; other disturbances (road accidents, signal failures, driver behavior variability) are out of scope. (f) Sim-to-real transfer is out of scope; the experimental claims are bounded to the calibrated SUMO microsimulation.

Chapter 3

Methods and Research Design

This chapter describes the development, training, and evaluation of a Multi-Agent Reinforcement Learning (MARL) model for dynamic bus scheduling under non-ideal conditions. The goal is a reproducible pipeline from a calibrated stochastic microsimulation of the EDSA Carousel corridor to measurable performance results under both ideal and non-ideal operating conditions.

3.1 Research Design

The study uses a **controlled comparative experimental design** with two between-condition factors and three response variables. The factors are control strategy (No Control, Forward Headway, Even Headway, MARL) and operating condition (ideal, non-ideal). The response variables are mean passenger waiting time, mean total travel time, and headway coefficient of variation. The non-ideal condition is not a single setting but a range of disturbance intensities, so controller performance is characterized as a function of disturbance magnitude rather than evaluated at a single point.

Each combination is replicated $N \geq 30$ times using independent Monte Carlo seeds, so the Central Limit Theorem applies to the run means [54]. The unit of analysis is one full simulated operating day on the defined EDSA Carousel sub-corridor. Random seeds and disturbance realizations are matched across control strategies within each disturbance level, so any performance gap between two controllers on the same run reflects the controller rather than the specific disturbance realization [48, 49]. Sampling is exhaustive within the simulation, so inclusion is defined at the disturbance level: only the modeled disturbances enter the non-ideal condition, and all others stay at their ideal values.

Throughout, disturbances play two distinct roles that must not be conflated. During **training**, the full range of stochastic disturbances is always active, the policy is trained under domain randomization [55] so that it learns to act under demand surges, traffic friction, heavy-tailed weather delays, and breakdowns. During **evaluation**, the same generators are used to define two test conditions (ideal and non-ideal) that probe the *already-trained* policy on disturbance realizations it did not see during training but which are drawn from the same generative process. The ideal/non-ideal split is therefore an **evaluation factor, not a training switch**: the agent is never trained on a disturbance-free corridor.

3.2 Methodology

3.2.1 Simulation Environment

The study uses a two-stage simulation architecture. **SUMO** (Simulation of Urban Mobility) [56] serves as the **calibration layer**, providing a microscopic representation of the EDSA Carousel corridor from which empirical parameters are extracted. A **custom Python environment built on the PettingZoo Agent Environment Cycle (AEC) API** [57] serves as the **training and Monte Carlo evaluation layer**, where MARL agents interact with a lightweight bus dynamics model parameterized by the distributions extracted from SUMO. SUMO is not used for training because its microscopic simulation of surrounding non-bus traffic is unnecessary for the agents’ reward function, which depends only on bus state, and because stepping that traffic at every control event across the full Monte Carlo budget would be computationally prohibitive. SUMO is therefore used once for calibration, and all training and evaluation runs are carried out in the lightweight Python environment.

PettingZoo’s AEC API is chosen over three alternatives. First, the single-agent Gymnasium interface [58] cannot natively represent multiple agents acting on independent events. Fitting multi-agent control into a single-agent API requires either treating the entire fleet as one composite agent, which breaks parameter sharing, or coordinating agents manually outside the API, which removes the benefit of using a standard interface. Second, PettingZoo’s parallel API assumes all agents step simultaneously [57], which does not match asynchronous bus control where only the bus arriving at a control stop has a decision to make. Third, a fully custom implementation was considered but rejected because it would sacrifice compatibility with established algorithm libraries and complicate reproducibility. AEC handles the asynchronous case by cycling through one agent at a time rather than stepping all agents in parallel [57]. This matches the approach used by Wang and Sun [28] and Rodriguez et al. [27], both of whom implemented the same asynchronous logic in custom event-driven simulators. Adopting AEC preserves their event-driven dynamics while providing a standardized interface compatible with parameter-sharing algorithm libraries.

SUMO is used for the calibration layer because it integrates with Python via the Traffic Control Interface (TraCI) and supports the GEH and RMSE validation protocols described later in this section. Ollero et al. [8] previously developed an AIMSUN microsimulation of the EDSA Busway corridor; this study implements an equivalent network in SUMO to take advantage of its open-source TraCI interface, while adopting the same calibration targets.

A pure-Python alternative without SUMO calibration was also considered but rejected: it would lack the empirically grounded per-segment travel-time distributions that the GEH-and-RMSE-calibrated SUMO model provides. The chosen approach uses each tool where it is strongest: SUMO for calibrated microscopic realism during the one-time calibration phase, and Python for fast event-driven Monte Carlo during training and evaluation. This keeps the experiment tractable on accessible hardware, including a single cloud-notebook GPU runtime. Rodriguez et al. [27] and Wang and Sun [28] adopt the same two-layer approach, using event-based custom simulators with calibration parameters supplied from empirical data.

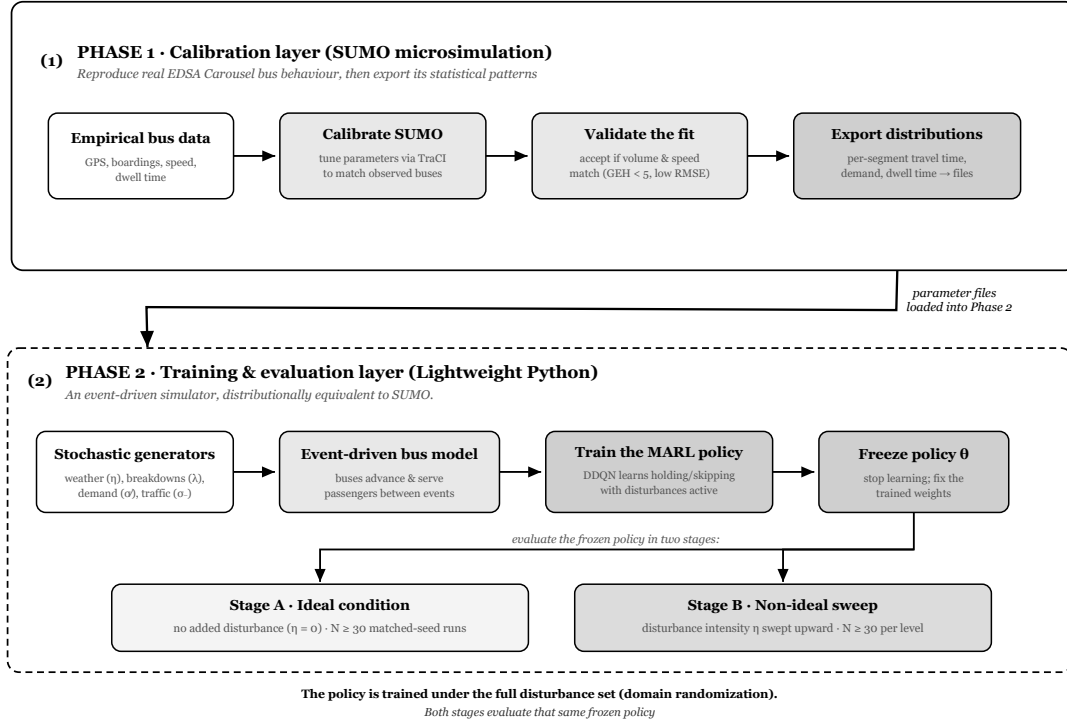


Figure 3.1: The two-phase simulation architecture.

The pipeline proceeds in two phases. First, SUMO is calibrated against empirical bus data, and the resulting per-segment travel time, demand, and dwell time distributions are saved. Second, the Python environment loads these distributions and runs a lightweight event-based bus simulator for MARL training and evaluation. Because the Python environment draws from distributions extracted from the calibrated SUMO model, its statistical behavior matches SUMO at the level of per-segment travel time, demand, and dwell time. It does not reproduce SUMO’s microscopic vehicle-to-vehicle interactions, but those interactions affect the reward function only through their aggregate effect on inter-stop bus travel time, which the calibrated distributions already capture. Rodriguez et al. [27] make the same argument: they trained in an event-based mesoscopic simulator parameterized by empirical corridor data.

3.2.2 Control-Stop Selection

The study operates on a sub-corridor of the EDSA Carousel route. Within the chosen sub-corridor, only a subset of stops is designated as *control stops* where MARL agents are permitted to act; the remainder are non-control stops that contribute stochastic dwell and delay behavior without agent intervention. Control stops are selected according to four criteria, following Rodriguez et al. [27] and the minimal-intervention principle of Wang and Sun [28]:

1. **Origin terminal.** The route origin is always a control stop, since it is the natural point at which initial headways are set before buses enter the corridor.
2. **Onset of high-demand segments.** The first stop of each high-demand segment is designated a control stop, so that the agent can act before demand-driven irregularity propagates downstream.

3. **Low through-volume preference.** Stops with substantial through-passenger volume (many onboard riders not alighting) are *avoided* as control stops, so that holding actions do not penalize large numbers of passengers already in transit.
4. **No adjacent control hubs.** Where two high-demand hubs are adjacent, only the upstream hub is designated, so that holding penalties do not compound across two consecutive control actions.

These criteria are data-driven: they specify *how* stops are selected from demand and through-volume statistics, without assuming which stops a particular dataset will yield. Once the operational data are processed (Section 3.2.5) and the per-stop demand and through-volume profiles are known, the criteria are applied to produce the control-stop list for the study sub-corridor.

3.2.3 Environment Model Validation

Before MARL training begins, the SUMO calibration layer is statistically verified to approximate real-world bus operation along the corridor. Two industry-standard calibration metrics are used: the GEH statistic for bus volume on the dedicated corridor, and the Root Mean Square Error (RMSE) for bus speed trajectories. The calibration is restricted to the bus corridor itself, since the agents' state and reward depend only on bus dynamics; surrounding mixed-traffic flows do not enter the Python environment.

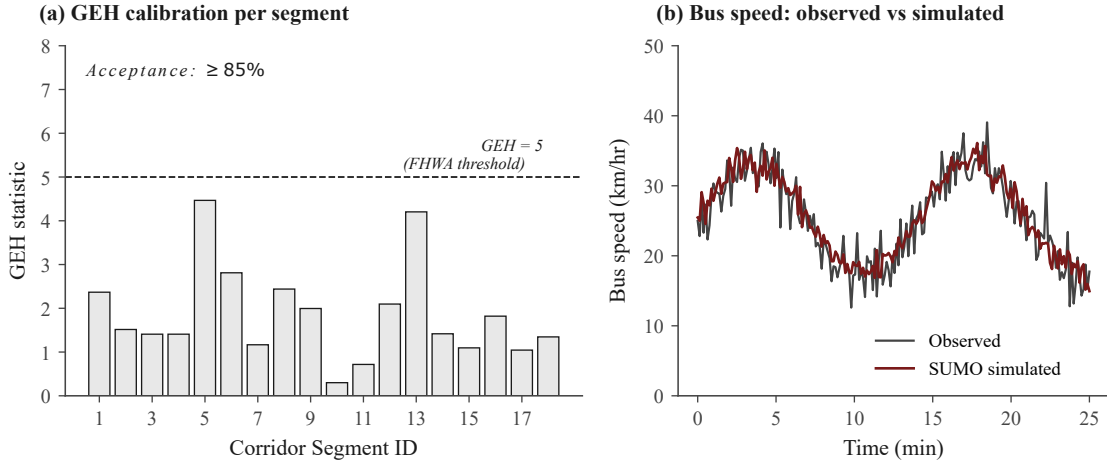


Figure 3.2: Illustrative format of the SUMO calibration output. (a) Per-segment GEH statistic against the FHWA acceptance threshold of $GEH < 5$; the corridor is accepted when at least 85% of segments fall below the line. (b) Simulated bus-speed trajectory against the empirical observation used in the RMSE minimization step. Values shown are illustrative only.

Bus Volume Validation via GEH Statistic

The GEH statistic, introduced for traffic-modelling acceptance testing in the UK Design Manual for Roads and Bridges [59], measures the discrepancy between simulated and observed hourly bus volumes on individual corridor segments:

$$GEH = \sqrt{\frac{2(M - C)^2}{M + C}} \quad (3.1)$$

where M is the hourly bus volume from the simulation and C is the observed hourly bus count from empirical operational data. The environment is accepted when $GEH < 5$ for more than 85% of corridor segments, following the FHWA Traffic Analysis Toolbox calibration criterion [60] as applied in recent SUMO calibration work by Patil et al. [50].

Speed Trajectory Calibration via RMSE

Volume calibration alone does not guarantee that buses move realistically. RMSE evaluates how closely simulated bus speed trajectories match empirical observations:

$$RMSE = \sqrt{\frac{\sum_{i=1}^n (x_{obs} - x_{model})^2}{n}} \quad (3.2)$$

where x_{obs} is the observed bus speed and x_{model} is the simulated bus speed [8]. To minimize RMSE, microscopic SUMO parameters like maximum desired speed and speed acceptance for the bus vehicle class are iteratively tuned to fit empirical speed profiles [8]. Once both the GEH threshold and RMSE minimization are achieved, the environment is treated as a validated representation of the operational scope.

Real-Time Data Incorporation

In a real-world deployment, the agent’s state vector at each control event would be drawn from existing transit-data infrastructure: forward and backward headways (h^- , h^+) from Automatic Vehicle Location (AVL) feeds; onboard and waiting passenger counts from Automatic Passenger Counters (APC) and Automatic Fare Collection (AFC) terminals; and environmental flags from public weather APIs and incident-management systems [28]. Within this study, the Python environment generates synthetic equivalents of these signals at each control event by querying the analytical bus model and combining outputs from the stochastic generators described below.

3.2.4 Operating Conditions

The study **evaluates** controllers under two distinct operating conditions. These conditions govern **evaluation only**; as detailed in the Training Protocol, the MARL policy is always trained with the full disturbance set active (domain randomization), and the two conditions below differ only in which disturbances are switched on when measuring the frozen, already-trained policy. The symbols used to define these conditions (η , σ_d , σ_s , and the breakdown rate λ) are collected in Table 3.1.

Ideal conditions. The corridor operates under the empirical baseline from the operational dataset, with no additional disturbance. The weather disturbance generator is disabled ($\eta = 0$), the breakdown generator is disabled ($\lambda = 0$), and the demand and traffic-speed scaling factors are at their baseline means ($\sigma_d = 0$, $\sigma_s = 0$). Each bus therefore encounters the empirical mean travel time and demand profile for its segment and time of day. The Python environment still injects normal day-to-day variability through the calibrated per-segment distributions, so the ideal condition is *stochastic but not disturbed* which comparable to a normal weekday with no incidents, severe weather, or surge events. This serves as the control condition for Stage A *evaluation*; it is not a training setting, since training always exposes the policy to the full disturbance range.

Non-ideal conditions. One or more of the four stochastic generators described in the next section is activated. The full non-ideal evaluation (Stage B) sweeps the weather disturbance intensity η across five values, with the breakdown generator ac-

tive alongside throughout. Demand scaling remains active under both conditions to preserve realistic passenger-arrival variability. The traffic-speed generator governs inter-stop travel-time variability under the ideal condition; under the non-ideal condition, the heavier-tailed weather generator replaces it as the source of travel-time stochasticity, since its coefficient of variation $\eta \geq 0.3$ already spans and exceeds the variability range that the traffic-speed scaling produces (Section 3.2.6). The two conditions therefore differ in which generator governs travel-time stochasticity and in whether the discrete breakdown generator is enabled.

Both conditions draw inter-stop travel time from distributions anchored to the same per-segment empirical baseline mean μ extracted during SUMO calibration; only the shape of variability changes between conditions. The non-ideal condition recovers the ideal condition as $\eta \rightarrow 0$, because the lognormal collapses to a point mass at μ when its coefficient of variation is zero. Performance differences between the two conditions therefore reflect a change in the variability profile of travel time, not a change in its mean or in the underlying network geometry.

Throughout this chapter, *ideal conditions* and *non-ideal conditions* refer to these operating states of the simulated environment. *Baseline controllers* refers separately to the three non-MARL control strategies (No Control, Forward Headway, Even Headway) against which the MARL policy is benchmarked. A condition is a state of the world; a controller is a choice of algorithm.

3.2.5 Data Processing

The MARL framework is trained and evaluated on a single stream of input data pre-processed before training: corridor operational data used to calibrate SUMO and to derive baseline parameters for the stochastic generators. A second stream, the synthetic state observations generated by the Python environment at runtime, is produced on-the-fly during training and consumed once per control event.

Required Datasets

- **Corridor bus operational data.** A per-trip record of EDSA Carousel bus operation along the study sub-corridor, collected over a continuous observation window of at least two weeks. The required fields are GPS-tracked vehicle location, boarding and alighting events, passenger occupancy, operating speed, and *dwelt time* at each stop, the dwelt time being the interval a bus spends stationary at a stop serving passengers, measured from the moment the doors open to the moment they close and the bus is ready to depart, exclusive of any holding time subsequently imposed by the controller. These records yield the empirical distributions of bus cruising speed, inter-stop travel time, and demand under ideal operating conditions, used both to calibrate SUMO and to define the baseline operating point of the stochastic generators. The baseline operating point for this study is established from a crowdsourced operational record collected from the EDSA Busway during July 2023 through the Safe-TravelPH mobile application.

Severe-weather conditions are not estimated from operational data in this study but are injected as a controlled experimental variable, with disturbance magnitudes anchored to validated literature values rather than to a corridor-specific severe-weather sample.

A short empirical observation window cannot reliably resolve the variance of a heavy-tailed severe-weather distribution at per-segment granularity, and a swept-

disturbance design characterizes controller performance across a range of conditions rather than at a single uncertain calibration point. Because the disturbance conditions are random by construction, they are not drawn from any single fixed dataset: each Monte Carlo run generates a fresh, independently seeded realization of demand, traffic-speed, weather, and breakdown disturbances. The single empirical dataset is required only to fix the *baseline* (ideal-condition) operating point against which these random disturbances are layered; the held-out validation split (Stage 3 below) confirms that the calibrated baseline generalizes beyond the records used to tune it. The full disturbance parameterization is described in Section 3.2.6.

Data Pre-Processing Pipeline

Pre-processing proceeds in three stages.

Stage 1: Cleaning. Trip records with missing GPS coordinates, missing timestamps, negative inter-stop times, or trips that fail integrity checks (for example, a later stop served before an earlier one) are dropped. Remaining records are normalized to a common time zone. Because the operational observation window is short (on the order of two weeks), the data cannot support separate weekend or holiday demand profiles — a window of this length yields too few weekend days and typically no holidays to estimate distinct calendars reliably. Records are therefore filtered to regular weekdays and binned by time-of-day, which is the calendar dimension that drives demand and travel-time variation within the modeled operating day. The restriction to weekday operation is recorded as a scope limitation.

Stage 2: Empirical distribution extraction. For each (segment, time-of-day) bin, the cleaned operational data yield the mean μ and standard deviation σ of inter-stop travel time under baseline operating conditions, producing the baseline coefficient of variation $CV_0 = \sigma/\mu$ used to anchor the disturbance generators. Boarding and alighting counts per (stop, time-of-day) bin yield the baseline demand profile. These statistics are serialized to disk as parameter files that the Python environment loads at initialization.

Stage 3: Train/validation split for calibration. The calibration data are split chronologically into a calibration set, used to tune SUMO parameters until the GEH and RMSE targets are met, and a held-out validation set, used to confirm that the calibrated environment generalizes beyond the tuning data.

The MARL agents are not trained on these raw datasets directly. They are trained on synthetic state vectors $s_{i,t}$ produced by the Python environment at each control event. Each $s_{i,t}$ is assembled at runtime by querying the analytical bus model and is consumed once by the policy network before being discarded. There is no persistent training dataset of state-action transitions on disk other than the standard replay buffer used by the learning algorithm.

3.2.6 Stochastic Disturbance Generators

Four stochastic generators inject variability into the Python environment. Generators (i) and (ii) follow the perturbation framework of Wang and Sun [28]; the weather generator’s heavy-tailed lognormal formulation follows Patil et al. [50]; the breakdown generator follows the rescheduling formulation of Cao et al. [39]. Table 3.1 collects the symbols used across this section and the MARL formulation that follows.

Table 3.1: Notation used in the stochastic generators and the MARL formulation.

Symbol	Definition	Domain / units
η	Weather disturbance intensity; directly sets the coefficient of variation of disturbed inter-stop travel time ($CV = \eta$). $\eta = 0$ disables the weather generator.	dimensionless
σ_d	Standard deviation of the per-episode passenger-demand scaling factor. $\sigma_d = 0$ fixes demand at its baseline mean.	dimensionless
σ_s	Standard deviation of the per-episode traffic-speed scaling factor. $\sigma_s = 0$ fixes cruising speed at its baseline mean.	dimensionless
λ	Rate parameter of the Poisson process governing bus breakdowns. $\lambda = 0$ disables breakdowns.	events/hour
μ	Baseline (empirical) mean inter-stop travel time for a given segment and time-of-day bin.	seconds
σ	Baseline standard deviation of inter-stop travel time.	seconds
CV_0	Baseline coefficient of variation, $CV_0 = \sigma/\mu$, used to anchor the disturbance generators.	dimensionless
μ_{ln}, σ_{ln}	Location and shape parameters of the lognormal weather-delay distribution (method-of-moments).	—
$h^-, \hat{h}^+$	Forward headway (gap to bus ahead) and estimated backward headway (gap to bus behind).	seconds
H_0	Desired (scheduled) headway.	seconds
$\alpha_{i,t}$	Holding-strength action of agent i , $\alpha_{i,t} \in \Omega = \{0.0, 0.1, 0.2, 0.3, 0.4\}$, scaled by ΔT .	dimensionless
ΔT	Maximum holding duration that $\alpha_{i,t}$ scales.	seconds
γ, β	Discount factor and event-based discount rate ($e^{-\beta t}$ replaces γ for random inter-event times).	dimensionless
θ	Weights (parameters) of the online neural network (shared policy).	dimensionless
θ^-	Weights (parameters) of the target neural network in DDQN.	dimensionless
N	Number of bus agents (also used for the Monte Carlo run count, $N \geq 30$, where noted).	count

Random Seed Control

All four generators draw from a single seeded pseudo-random number stream maintained by the Python NumPy library. At the start of each Monte Carlo run, a master seed is set in the simulation driver script, which initializes NumPy’s BitGenerator. For each (disturbance level, control strategy) cell, the list of $N \geq 30$ seeds is fixed in advance and stored as a configuration file in the project repository. Within a disturbance level, the same seed list is reused across all four control strategies, so the demand trace, disturbance realization, and breakdown timing for run k are identical regardless of which controller is active. This matching enables paired statistical comparisons: any difference in performance metrics between two controllers on run k is attributable to the controller rather than to the disturbance realization [48]. Seeds across runs are independent, and the choice $N \geq 30$ ensures that the Central Limit Theorem applies to the run-mean of each response variable.

Passenger Demand

The baseline empirical transit demand is perturbed each episode by a scaling factor sampled from $\mathcal{N}(1, \sigma_d^2)$, clipped to $[1, 3]$, following Wang and Sun [28]. The asymmetric clip focuses the test on demand surges rather than symmetric variation, since demand drops produce lightly loaded conditions that do not stress-test the controller. The upper bound of 3 corresponds to roughly a tripling of baseline boarding rates, spanning the range observed during major event let-outs and severe-weather mode shifts. Sampling occurs at the start of each simulation run, producing varied demand profiles across episodes.

Traffic Delays

Mean cruising speed between stops is adjusted dynamically using a scaling factor drawn from $\mathcal{N}(1, \sigma_s^2)$, clipped to $[0.8, 1.2]$, representing typical daily congestion friction [28]. This generator provides the baseline stochastic variability in inter-stop travel time when the weather generator is inactive. When the weather generator is active ($\eta > 0$), the heavier-tailed lognormal described below replaces this scaling as the source of travel-time variability: the lognormal coefficient of variation $\eta \geq 0.3$ used in the non-ideal sweep already exceeds the $\pm 20\%$ range that traffic-speed scaling produces, so the two generators do not compose on the same segment. The traffic-speed generator’s empirical role persists through the per-segment baseline mean μ to which both generators are anchored.

Weather-Induced Anomalies

Severe-weather disturbances are injected as a controlled experimental variable rather than estimated from a fixed empirical sample.

The controlled variable in this generator is the coefficient of variation of inter-stop travel time, not a meteorological quantity. Severe weather is the motivating phenomenon, but the variable that enters the simulator and that the controller observes is the travel-time distribution itself. The lognormal is chosen to model the shape that any such heavy-tailed disruption produces on per-segment travel time, regardless of its meteorological label. The disturbance intensity sweep is therefore read as a span of travel-time variability magnitudes, not as a sweep across named weather categories.

Inter-stop travel time under disturbance is sampled from a Lognormal distribution, which captures the right-skewed, heavy-tailed delays observed in urban traffic under inclement weather [50]. The lognormal is chosen over a Gaussian distribu-

tion, which is symmetric and cannot represent heavy right tails, and over Gamma or Weibull distributions, which can produce heavy tails but lack the direct empirical validation against urban traffic data that Patil et al. provide for the lognormal form. The distribution’s mean is the baseline inter-stop travel time μ from the operational data, and its variability is set by a disturbance intensity parameter η , which directly specifies the coefficient of variation:

$$CV = \eta \quad (3.3)$$

The lognormal shape and scale parameters follow the method-of-moments derivation:

$$\sigma_{ln} = \sqrt{\ln(\eta^2 + 1)} \quad (3.4)$$

$$\mu_{ln} = \ln(\mu) - \frac{\sigma_{ln}^2}{2} \quad (3.5)$$

Travel time is drawn as $T \sim \text{LogNormal}(\mu_{ln}, \sigma_{ln})$. Patil et al. [50] validated this parameterization against INRIX freeway data via the Kolmogorov-Smirnov test, reporting a close fit at the highest variability level they tested ($KS = 0.036$, $p = 0.94$ at $CV = 1.0$).

The disturbance intensity η is swept across five values, $\eta \in \{0.0, 0.3, 0.6, 1.0, 1.3\}$, chosen to span and slightly exceed the empirically validated CV range of Patil et al. [50], who evaluate scenarios at $CV \in \{0.1, 0.3, 0.5, 0.7, 1.0\}$. The basis for each swept value is as follows. $\eta = 0$ disables the generator and recovers the ideal condition. $\eta \in \{0.3, 0.6\}$ fall inside the validated range, representing moderate travel-time variability. $\eta = 1.0$ corresponds to the upper end of Patil et al.’s validated range and is adopted here as the nominal severe-disturbance operating point. $\eta = 1.3$ extends beyond the validated range as an extrapolated stress test; results at this level are interpreted as a probe of controller behavior under extreme conditions rather than as a calibrated scenario.

The mapping from η to a specific named weather severity (e.g. “moderate rain” versus “typhoon-grade”) is not established by Patil et al., whose CV values are not labeled by weather category. The η sweep should therefore be read as a span of travel-time variability magnitudes rather than as calibrated meteorological intensities. Reporting controller performance across this sweep characterizes degradation as a function of disturbance magnitude, which is methodologically stronger than evaluating at a single point.

Bus Breakdowns

Unlike the perturbations above, a bus breakdown is a *discrete event* that removes a bus from active simulation [39, 46]. Breakdowns are triggered at random times sampled from a Poisson process with a configurable rate λ (Table 3.1). When a breakdown occurs at bus b_k , b_k is removed from the active agent set, leaving an enlarged headway between the trailing bus b_{k-1} and the leading bus b_{k+1} . The passengers that b_k would have served accumulate at the stops ahead of b_{k-1} , so the trailing bus inherits this residual demand as it advances.

A breakdown does not override any agent’s policy; it changes the **state** the surviving agents observe and the **reward landscape** they face. The removal of b_k is encoded in the downstream incident/breakdown flag of the trailing agent’s observation (Section 3.2.7), and the inherited residual demand sharply raises the

headway-irregularity and dwell-time penalties that the trailing agent’s reward already captures. Under these conditions, the stop-skipping action, which is part of the agent’s normal action space (Section 3.2.7) becomes strongly advantageous. Skipping a heavily loaded downstream stop lets b_{k-1} shed onboard passengers (alighting permitted, boarding suppressed) and close the gap to b_{k+1} before bunching sets in. The environment therefore *induces* the recovery behavior through the state and reward signal, and the agent must *learn* to select it; the action is never injected by the environment. This keeps the breakdown response consistent with decentralized execution and the Markov-game formulation, in which every action originates from π_θ .

The Poisson assumption is a tractable approximation for the single-day simulation horizon, over which vehicle age, maintenance schedule, and mileage since last service can be treated as constant.

3.2.7 MARL Architecture and MDP Formulation

This subsection builds the formal learning framework in two steps. It first introduces the Bellman recursion that underlies value-based reinforcement learning, then specifies the Markov game tuple, the per-agent observation and action spaces, the reward-function design, and the proposed learning algorithm. The justification for choosing a multi-agent formulation over a single-agent alternative is presented in Chapter 1.

Bellman Foundation

At the core of reinforcement learning is the dynamic programming principle established by Bellman [61]. The agent’s objective is to maximize the expected sum of discounted future rewards. The state-action value function $Q^\pi(s, a)$, denoting the expected return from taking action a in state s and following policy π thereafter, satisfies the Bellman optimality equation:

$$Q^*(s, a) = \mathbb{E} \left[r(s, a) + \gamma \max_{a'} Q^*(s', a') \mid s, a \right] \quad (3.6)$$

where s' is the state reached after applying action a , $r(s, a)$ is the immediate reward, and $\gamma \in [0, 1)$ is the discount factor. This equation is relevant here for two reasons. First, it shows that a long-horizon control policy can be learned by recursively backing up local one-step rewards, making it computationally feasible to evaluate the downstream effects of a holding or skipping decision without enumerating every future stop on the route. Second, in the asynchronous bus-control setting, the time between two consecutive control events is itself random. Following Bradtke and Duff [62], the discount factor in Eq. (3.6) is replaced by an event-based discount $e^{-\beta t}$ that scales with the elapsed time between events. Each agent’s learning target is therefore an event-adjusted version of Eq. (3.6), not a fixed-tick MDP target. By convention throughout this chapter, γ denotes the nominal discount factor as it appears in the canonical Bellman and DDQN equations, while $e^{-\beta t}$ is the event-based discount actually applied in the asynchronous setting; wherever γ appears in a value-function equation, it is instantiated by $e^{-\beta t}$ at runtime.

Markov Game Formulation

Following Littman [63], the bus fleet control problem is formulated as a multi-agent extension of a Markov Decision Process, a Markov game defined by the tuple $G = \langle N, S, A, P, R, \gamma \rangle$:

- N : number of bus agents.
- S : joint state space.
- A : joint action space.
- P : state-transition function $P(s_{t+1} \mid s_t, a_t)$.
- R : reward function.
- $\gamma \in [0, 1)$: discount factor.

Because bus control is asynchronous and event-driven, agents act only upon arrival at a control stop. The formulation maintains a per-agent local transition $P_i : S_i \times A_i \rightarrow S_i$, with S_i the local observation of agent i [27, 28].

Agents (N)

The decision-making agents are the individual bus units $i \in N$ operating along the corridor. To avoid combinatorial blow-up and to keep the system invariant to fleet size, all agents share a single neural-network policy under a **parameter-sharing** architecture and act on their own local observations at deployment [27, 28, 32]. Parameter sharing means that one set of network weights serves every bus; each bus computes its own action from its own observation, but learning updates from any bus improve the policy for all.

State Space and Local Observations (S_i)

The local observation $s_{i,t}$ at agent i on arrival at a control stop is a vector composed of:

- **Spatial location:** index of the current control stop.
- **System regularity:** forward headway h^- and estimated backward headway $\hat{h}^+$, both available from AVL feeds in deployment [28].
- **Passenger demand:** onboard passenger count and estimated number of passengers waiting at the stop, available from APC/AFC systems [28].
- **Environmental flags:** encoded indicators for the current disturbance intensity and any active downstream incident or breakdown.

Action Space (A_i)

The agent's action $a_{i,t} = \pi_\theta(s_{i,t})$ combines two discrete components, following Rodriguez et al. [27]:

- **Holding control:** a holding-strength parameter $\alpha_{i,t} \in \Omega = \{0.0, 0.1, 0.2, 0.3, 0.4\}$, multiplied by a maximum holding duration ΔT to yield seconds held; $\alpha_{i,t} = 0$ means no holding.
- **Stop-skipping:** a binary decision to bypass the next stop, triggered when the bus is severely delayed and near capacity, allowing headway recovery. Skipping is disallowed at the origin terminal and is not permitted if the immediately preceding trip already skipped that stop, so no passenger is denied service twice in a row.

The full action set is the Cartesian product of these two components: $|A_i| = 5 \times 2 = 10$ discrete actions per control event. A continuous holding parameter $\alpha \in [0, 1]$ was considered, following Wang and Sun [28], but rejected for three reasons. First, continuous actions require actor-critic algorithms, whose training instability compounds across the swept-disturbance evaluation budget. Second, Rodriguez et al. [27] showed that a 5-bin discretization of α achieves combined holding-and-skipping control on a comparable corridor without measurable loss of performance versus continuous formulations. Third, real driver compliance with second-level holding instructions is itself coarse [27], so continuous precision in α is not meaningful at deployment.

Reward Function (R) and Objective

The infinite-horizon objective of agent i is the expected cumulative event-discounted return

$$R_i = \mathbb{E} \left[\sum_{k=0}^{\infty} e^{-\beta t_k} r_{i,t+k} \right], \quad (3.7)$$

where t_k is the elapsed time to the k -th future control event, consistent with the event-based discount introduced earlier.

The **structure** of the per-event reward $r_{i,t+k}$ — its component terms and the operational priorities they encode — is defined in this chapter. Its **exact weighting and sensitivity analysis** are a research objective of this study (Specific Objective 2, Expected Output 2.1), to be finalized during implementation rather than inherited from prior work. This separation is deliberate: the reward *form* is grounded here, while the tuning of its coefficients is treated as an empirical question. The reward formulation must balance three competing operational priorities: (i) headway regularity, since bunching is the primary failure mode of bus corridors; (ii) passenger waiting time at served stops, which is the most direct passenger-experience metric; and (iii) a penalty term that discourages degenerate skipping behavior, since an unconstrained skip action could otherwise reduce headway variance by refusing to serve passengers. Existing MARL bus-control reward formulations span a range of trade-offs among these priorities, from headway-coefficient-of-variation forms for holding-only action spaces [28, 31] to passenger-time forms for combined holding-and-skipping action spaces [27]. This study defines the reward *structure* for the hybrid action space, the three component terms above, and treats their relative weighting, plus a sensitivity analysis over those weights, as the implementation-phase deliverable (EO 2.1). The component structure is fixed; the coefficients are not yet finalized.

Proposed Learning Algorithm

The fully discrete action space ($|A_i| = 10$) makes value-based learning a natural fit. This study adopts **Double Deep Q-Network (DDQN) [64] with parameter sharing under centralized training and decentralized execution (CTDE)**. CTDE means the policy is trained with access to information from all agents but each bus acts on its own local observation alone at deployment, so the trained system does not require a centralized real-time controller. The choice follows Rodriguez et al. [27], who used the same DDQN-based framework for cooperative holding-and-skipping control on a comparable corridor.

DDQN is chosen over three categories of alternatives. *Plain DQN* [65] overesti-

mates Q -values in noisy environments [48, 64], which the high-variance disturbance regime in this study would amplify. DDQN’s double estimator addresses this by decoupling action selection from evaluation [64]. *Actor-critic methods such as MAPPO and MADDPG* are compatible with continuous action spaces but unnecessary given the discrete action-space decision above; they would also introduce training instability that compounds across the swept-disturbance evaluation budget. *Distributional methods such as Wang and Sun’s IQNC-M* [28] model the full return distribution rather than its expectation and offer strong performance on this problem class, but their meta-learning training procedure adds implementation risk beyond the scope of a thesis-scale project; IQNC-M is noted as a comparison target for future work.

Two enhancements from the recent bus-control literature will be tested during implementation: extending agent awareness to include neighboring-trip information in the observation and reward, as in Rodriguez et al.’s DDQN-HA variant [27], and credit assignment via inductive graph learning, as in Wang and Sun’s CAAC framework [28]. The final architecture will be selected based on training stability and validation-set passenger waiting time after a fixed training budget.

Double Deep Q -Network Foundation

The Bellman optimality equation defines the value $Q^*(s, a)$ that an ideal controller would assign to each state-action pair, but it cannot be solved exactly for a state space as large as the bus-control problem, where the observation is a continuous vector of headways, loads, and queues. Two approximations bridge this gap. Q -learning [61, 63] replaces the exact solution with an iterative update that nudges a stored estimate $Q(s, a)$ toward the one-step Bellman target $r + \gamma \max_{a'} Q(s', a')$ after each transition. The Deep Q -Network (DQN) [65] replaces the lookup table with a neural network $Q(s, a; \theta)$, so that the value function generalizes across the continuous state vectors the bus environment produces. DQN trains θ by minimizing the temporal-difference loss between the network’s prediction and a target computed from a periodically frozen copy of the network, with transitions drawn from an experience replay buffer to decorrelate consecutive samples.

DQN has a known issue: it systematically overestimates action values [64]. The cause is the single max operator in the target. Because the same network both selects the next action ($\arg \max_{a'}$) and evaluates it ($\max_{a'} Q$), noise in value estimates is amplified the max preferentially picks whichever action happens to be overvalued, and that inflated value propagates into the target. In a low-variance environment this bias is manageable, but the swept-disturbance regime in this study generates heavy-tailed returns that feed the overestimation mechanism directly.

The Double Deep Q -Network (DDQN) [64] removes this bias by decoupling action selection from action evaluation. The online network (weights θ) selects the next action, but a separate target network (weights θ^-) evaluates that action’s value:

$$y^{\text{DDQN}} = r(s, a) + \gamma Q\left(s', \arg \max_{a'} Q(s', a'; \theta); \theta^-\right), \quad (3.8)$$

For the overestimation to persist, both networks would have to overvalue the same action simultaneously, which is less likely than a single network doing so alone. The network is trained by minimizing the squared temporal-difference error:

$$\mathcal{L}(\theta) = \mathbb{E}_{(s, a, r, s') \sim \mathcal{D}} \left[\left(y^{\text{DDQN}} - Q(s, a; \theta) \right)^2 \right], \quad (3.9)$$

where $\mathcal{D}$ is the experience replay buffer and the target weights θ^- are held fixed

for a number of updates (or slowly tracked toward θ) so that the regression target does not shift at every step.

Three properties make DDQN appropriate for this study. First, the discrete action space ($|A_i| = 10$) is the setting value-based methods handle well: the $\arg \max$ in Eq. (3.8) is a finite enumeration over ten options. Second, the double estimator addresses the overestimation that high-variance disturbance returns would otherwise amplify. Third, DDQN inherits DQN’s experience-replay and target-network machinery, which provides sufficient training stability without the auxiliary mechanisms that actor-critic methods require; this point is discussed further in Section 3.3. The event-based discount $e^{-\beta t}$ replaces the fixed γ in Eq. (3.8) when the target is computed in the asynchronous setting, so the DDQN target inherits the same event-adjustment as the underlying Bellman recursion. The combination of DDQN with parameter sharing and the CTDE scheme is specified in the Proposed Learning Algorithm subsection.

Training and Execution Protocol

The learning process follows the standard reinforcement-learning feedback loop under the asynchronous, event-driven bus-control setting and the CTDE scheme described above. The loop, repeated at every control event during training, proceeds as follows:

1. **Observe.** When bus agent i arrives at a control stop, the Python environment assembles its local observation $s_{i,t}$ (Section 3.2.7) from the analytical bus model and the active stochastic generators.
2. **Act.** The shared policy network selects an action $a_{i,t}$ (holding strength and skip decision) from $s_{i,t}$ using an ϵ -greedy rule over the estimated Q -values during training, and greedily ($\arg \max_a Q$) at evaluation.
3. **Transition and reward.** The environment applies $a_{i,t}$, advances the simulation to the next control event, and returns the event-discounted reward $r_{i,t}$ (Section 3.2.7).
4. **Store.** The transition $(s_{i,t}, a_{i,t}, r_{i,t}, s_{i,t'})$, where t' is the agent’s next control event, is written to a shared experience replay buffer common to all agents under parameter sharing.
5. **Update.** Mini-batches sampled from the replay buffer update the shared online network by minimizing the DDQN temporal-difference loss against a slowly-updated target network; experience from every bus updates the single shared parameter set.

This loop is the feedback mechanism that distinguishes the learned controller from the fixed-rule baselines, which compute a holding time directly from the current headways with no update step.

The protocol is split into a training phase and an execution (evaluation) phase:

- **Centralized training phase.** The loop above is run for a fixed episode budget with the stochastic disturbance generators active, so that the policy encounters demand surges, traffic-speed friction, weather delays, and breakdowns during training. A policy that never sees a breakdown or heavy-tailed delay during learning has no basis for responding to one at deployment. This follows the

domain-randomization approach of Tang et al. [55], who randomize demand and schedule variables during training to obtain policies that generalize to real-world traffic variability. Training uses system-wide information where it stabilizes learning (e.g. a shared reward and shared buffer), and experience from all buses contributes to one shared network.

- **Decentralized execution phase.** The trained network’s weights are frozen. During Monte Carlo evaluation (Stages A and B), each bus acts independently and greedily on its own local observation $s_{i,t}$ alone, with no centralized controller and no further weight updates. The fixed policy is evaluated across both the ideal condition (Stage A) and the swept non-ideal condition (Stage B), so that its behavior is measured on disturbance realizations it did not see during training but which are drawn from the same generative process.

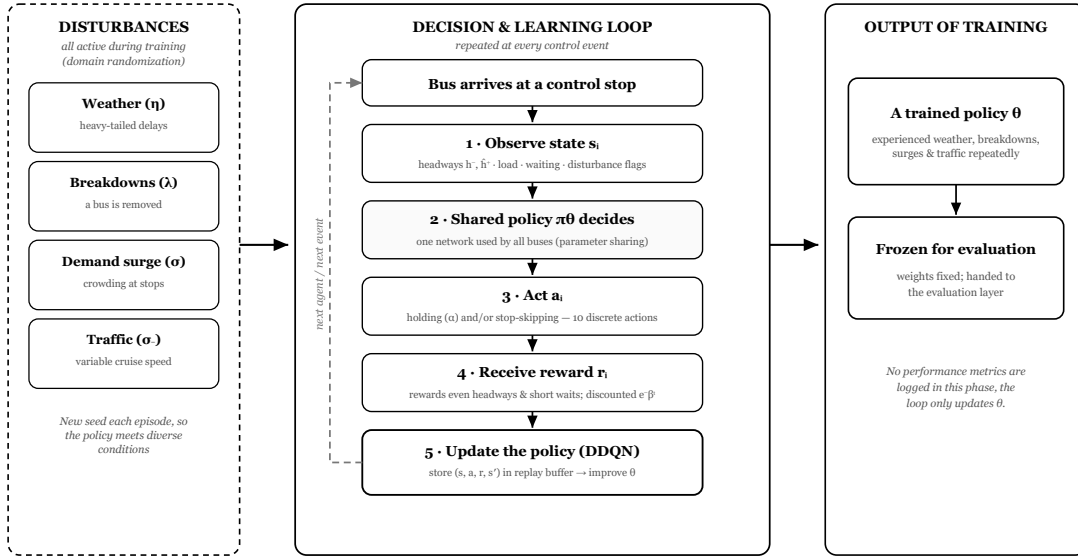


Figure 3.3: The asynchronous agent-environment cycle during training.

Because heavy-tailed returns can destabilize value-based learning if injected without safeguards [48, 49], training under disturbance is paired with the stabilizers discussed in Section 3.3: DDQN’s double estimator, reward clipping, and slow target-network updates. If training under the full disturbance set proves unstable at the highest intensities despite these measures, a curriculum that ramps disturbance intensity over the course of training [55] is retained as a fallback.

3.2.8 Baseline Controllers for Comparison

The MARL policy is benchmarked against three baseline controllers: **No Control (NC)**, **Forward Headway (FH)**, and **Even Headway (EH)**. These are the standard benchmark set in the MARL bus-control literature; both Wang and Sun [28] and Rodriguez et al. [27] use this trio. Using the same baselines makes the results directly comparable to published work.

No Control (NC)

No holding or skipping is applied. Each bus dwells only for the time required to serve boarding and alighting passengers, then departs immediately. NC serves as the lower bound of the comparison: it isolates the natural corridor behavior without any regularization. If MARL cannot beat NC, the controller is doing more harm than good. NC also provides the reference point for measuring the severity of bus bunching.

Forward Headway (FH)

The classical holding rule introduced by Daganzo [18]. When a bus arrives at a control stop, its holding time is:

$$d_{FH} = \max\{0, \bar{d} + g(H_0 - h^-)\} \quad (3.10)$$

where H_0 is the desired (scheduled) headway, h^- is the forward headway (time gap to the bus ahead), $\bar{d}$ is the average slack delay at equilibrium, and $g > 0$ is a control gain parameter. If a bus is running closer to the bus ahead than scheduled ($h^- < H_0$), it holds for an additional period proportional to the gap deficit; if it is already at or beyond the desired spacing ($h^- \geq H_0$), it does not hold. Parameters $\bar{d}$ and g follow Daganzo’s original values. FH represents local, single-agent reactive control: each bus reacts only to information about the bus immediately ahead.

Even Headway (EH)

A more globally aware rule that holds buses to equalize the forward and backward gaps simultaneously, following Rodriguez et al. [27]:

$$T_{EH} = \max\left\{\min\left[\frac{\hat{h}^+ - h^-}{2}, 0.4 H_0\right], 0\right\} \quad (3.11)$$

where h^- is the forward headway, $\hat{h}^+$ is the estimated backward headway (time gap to the bus behind), and $\frac{\hat{h}^+ - h^-}{2}$ is the holding time that would split the difference evenly between the two gaps. The hold is capped at $0.4 H_0$ (40% of the scheduled headway), the bound Rodriguez et al. used to prevent excessive intervention. This cap matches the maximum value in the MARL action set $\Omega = \{0, 0.1, 0.2, 0.3, 0.4\}$, so EH and MARL are constrained to the same maximum hold and the comparison is fair. EH represents globally-aware reactive control: it uses information about both neighbors but follows a fixed rule rather than a learned policy.

Baseline Comparisons

The three controllers span the comparison space. NC tests whether intervention helps at all. FH tests whether MARL outperforms classical local reactive control. EH tests whether MARL outperforms classical global reactive control and EH is the stronger of the two classical baselines, consistently reported as the best non-RL controller in the literature [27, 28]. The acceptance criterion for the MARL policy in Stage A (Section 3.2.10) is therefore stated relative to EH: matching or beating EH is the meaningful test.

This three-way comparison also allows the study to attribute MARL’s advantage. If MARL beats NC but not FH or EH, the benefit is from intervening, not from learning. If MARL beats FH but not EH, the benefit is from global awareness, not from learning. Only if MARL beats EH does the benefit come from the learned policy itself. The same logic applies in Stage B: the disturbance-handling claim is meaningful only if MARL’s advantage over EH grows with η , since both controllers see the same disturbance realizations.

State-of-the-art MARL methods such as CAAC and IQNC-M [28] were considered as additional baselines but excluded. Including them would shift the research question from “does MARL help on EDSA?” to “which MARL variant is best?”, a different question requiring different experimental data.

3.2.9 Data Analysis Methods

For each (control strategy, disturbance level) cell, $N \geq 30$ independent Monte Carlo runs are executed using matched random seeds across strategies. Three response variables are logged per run: mean passenger waiting time, mean total travel time, and headway coefficient of variation. The choice $N \geq 30$ ensures that the Central Limit Theorem applies to the run-mean of each response variable [54].

Statistical significance of performance differences between controllers will be assessed at a significance level of $\alpha = 0.05$. Because the same random seeds are reused across controllers within a disturbance level, comparisons between two controllers at the same disturbance level are naturally paired: the only difference between two runs is the controller, not the disturbance realization. Paired-sample tests will therefore be used for within-disturbance-level comparisons; unpaired tests will be used where seed matching does not apply (for example, comparing the same controller across different disturbance levels). Multiple-comparison correction will be applied across the four controllers to control the family-wise error rate.

For non-ideal disturbance levels, performance degradation is computed as the percentage change in each response variable relative to the same controller’s $\eta = 0$ ideal-condition result. Confidence intervals on this degradation ratio will be computed by bootstrap resampling, since ratios of sample means are not guaranteed to be normally distributed and may be skewed under high-disturbance conditions [66]. The specific hypothesis tests, normality checks, multiple-comparison procedure, and bootstrap parameters will be selected during the implementation phase in consultation with the research supervisor, following recent guidance on reinforcement-learning evaluation [48, 49]. Analysis is performed in Python using NumPy, SciPy, and pandas; raw run logs and analysis notebooks are version-controlled for reproducibility.

3.2.10 Evaluation Methods

Stage A: Ideal-condition evaluation. The weather and breakdown generators are disabled; demand and traffic-speed scaling are fixed at baseline means ($\eta = 0$). The trained MARL policy and the three baselines are each run for $N \geq 30$ Monte Carlo iterations with matched seeds. The acceptance criterion is twofold: (i) mean passenger waiting time no worse than EH (no statistically significant degradation at $p < 0.05$ with multiple-comparison correction), and ideally a statistically significant improvement; and (ii) a statistically significant reduction in headway coefficient of variation relative to NC. Under ideal conditions EH is already near-optimal, so parity with EH is an acceptable Stage A outcome. Outperforming EH is the target but not required to pass Stage A, since the main contribution of this study is the characterization of performance under disturbance in Stage B.

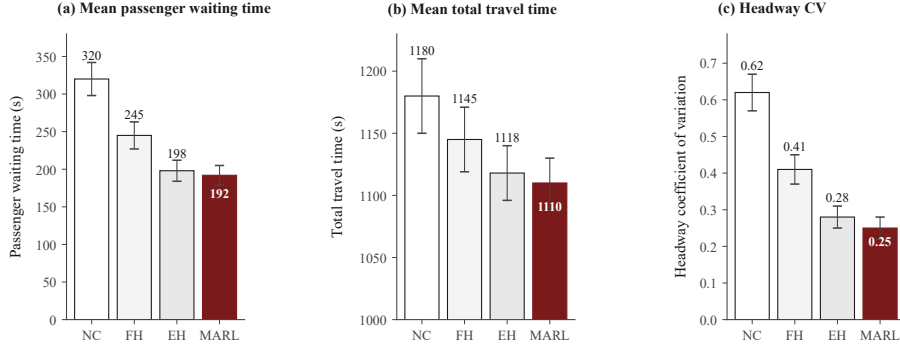


Figure 3.4: Illustrative Output of the Stage A ideal-condition evaluation output, comparing the four controllers (NC, FH, EH, MARL) across the three response variables: (a) mean passenger waiting time, (b) mean total travel time, and (c) headway coefficient of variation. Error bars represent 95% confidence intervals over $N \geq 30$ Monte Carlo runs with matched seeds. Numerical values shown are illustrative and do not reflect measured results.

Stage B: Non-ideal-condition evaluation. The trained MARL policy and the three baselines are evaluated across the swept disturbance range $\eta \in \{0.3, 0.6, 1.0, 1.3\}$, with the breakdown generator active at each level. For each $(\eta, \text{controller})$ cell, $N \geq 30$ Monte Carlo iterations are run with matched seeds. Reported outputs are: (a) mean waiting time, travel time, and headway CV as functions of η for each controller; (b) percentage degradation at each η relative to the controller’s own ideal-condition result; (c) the disturbance level at which MARL’s advantage over the best-performing baseline becomes statistically significant; and (d) the degradation slope, defined as the rate of waiting-time increase per unit increase in η , with smaller slopes indicating greater resilience. An ablation is also run with each disturbance enabled alone (demand-only, weather-only, breakdown-only) at $\eta = 1.0$ to attribute degradation to disturbance type. The MARL policy is reported as performing well under disturbance if its mean waiting time stays below that of the best-performing baseline across the full sweep, with statistical significance under the same correction.

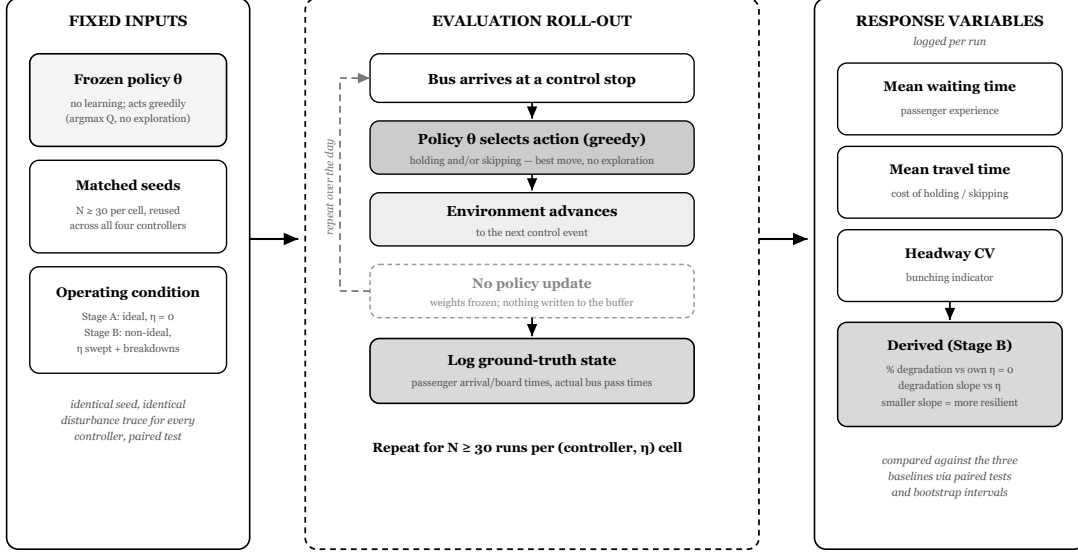


Figure 3.5: The Monte Carlo evaluation layer. The frozen policy θ selects the highest-valued action at each control stop without exploration or weight updates. Matched seeds ensure that every controller faces the same disturbance realization, so performance differences reflect the control strategy alone. Three response variables are recorded per run: mean passenger waiting time, mean total travel time, and headway coefficient of variation.

3.3 Methodological Challenges

Three challenges shape the experimental design. The design is scoped to remain feasible on accessible hardware (a single cloud-notebook GPU runtime), and a tiered scope plan (stated at the end of this section) guarantees a defensible minimum deliverable even if the full sweep cannot be completed.

Q-value overestimation under heavy-tailed returns. Value-based methods such as DQN overestimate Q-values [64], and this bias grows when returns are heavy-tailed, as they are under high-intensity disturbance [48, 49]. The DDQN double estimator addresses this by decoupling action selection from evaluation [64]. Because disturbances are active during training, reward clipping and target-network soft updates are used as additional stabilizers throughout, not only at the highest intensities.

Computational cost. The evaluation budget follows directly from the experimental factors. The full η grid spans five values $\{0, 0.3, 0.6, 1.0, 1.3\}$, with $\eta = 0$ serving as the Stage A ideal condition and the remaining four levels constituting the Stage B non-ideal sweep. Stage A runs four controllers at $N \geq 30$ each, giving $4 \times 30 = 120$ runs. Stage B runs four controllers across four non-ideal levels at $N \geq 30$ per cell, giving $4 \times 4 \times 30 = 480$ runs. The single-disturbance ablation (Section 3.2.10) adds three conditions (demand-only, weather-only, breakdown-only) across the same four controllers at $N \geq 30$, giving $3 \times 4 \times 30 = 360$ runs, for a total of approximately 960 evaluation runs. Two decisions keep this tractable. First, SUMO

is restricted to one-time calibration; all Monte Carlo runs execute in the lightweight Python environment. Second, evaluation runs are CPU-bound and parallelizable across random seeds, so training is the only phase requiring sustained GPU time.

Timeline feasibility. The full evaluation suite of approximately 960 runs is expected to require on the order of single-digit hours of single-core CPU time, given the event-driven simulation rates reported in comparable work [27, 28]; parallelizing across seeds on a multi-core runtime fits the full sweep within a single working session. MARL training is sized to the episode budget reported by Rodriguez et al. [27], who train for $500 + 300 = 800$ episodes in two phases. The total training budget, including hyperparameter tuning and reward-weight iteration (EO 2.1), is projected at approximately 30 hours of T4-class GPU time, which fits within standard cloud-notebook allocations without institutional HPC resources. These time figures are planning estimates to be confirmed empirically during implementation rather than measured values. A tiered scope plan is maintained: the minimum deliverable is Stage A ($\eta = 0$) plus a two-level non-ideal sweep at $\eta \in \{0.6, 1.0\}$; the full four-level non-ideal sweep and ablation series are added if calibration and training proceed on schedule.

Bibliography

- [1] L. Paget-Seekins and M. Tironi, “The publicness of public transport: The changing nature of public transport in latin american cities,” *Transport Policy*, vol. 49, pp. 176–183, 2016.
- [2] United Nations Department of Economic and Social Affairs, Population Division, “World urbanization prospects: The 2018 revision,” Tech. Rep. ST/ESA/SER.A/420, United Nations, New York, NY, USA, 2019.
- [3] R. Cervero, *Informal Transport in the Developing World*. Nairobi, Kenya: UN-Habitat, 2000.
- [4] S. R. D. Castro, C. A. C. Tadde, and B. G. Carcellar III, “Spatiotemporal analysis of traffic accidents and traffic congestion along commonwealth avenue using crowdsourced waze traffic data,” *ISPRS Annals of the Photogrammetry, Remote Sensing and Spatial Information Sciences*, vol. X-5/W4-2025, pp. 165–172, 2026.
- [5] S. F. Chua, *Stress Among Bus and Personal Drivers in the Philippines*. PhD thesis, Queensland University of Technology, Brisbane, QLD, Australia, 2026.
- [6] Philippine Star (Department of Transportation data), “EDSA busway ridership hits over 66 million in 2025.” Philstar.com news report citing Department of Transportation (DOTr) records, Jan. 2026. Reports 2025 annual ridership of 66.67 million (up from 63.02 million in 2024), average daily ridership of approximately 182,655, and a single-day peak of 321,186 passengers in April 2025.
- [7] S. M. Gaspay, N. C. Tiglao, M. A. Tacderas, N. J. Tolentino, and A. C. Ng, “Reforms in metro manila’s bus transport system hastened by the covid-19 pandemic: A policy capacity analysis of the edsa busway,” *Research in Transportation Economics*, vol. 100, p. 101305, 2023.
- [8] L. J. Ollero, K. Vergel, and N. C. C. Tiglao, “A microsimulation model of an exclusive bus lane: The case of edsa busway,” *Philippine Transportation Journal*, vol. 7, no. 2, 2024.
- [9] N. C. C. Tiglao, E. Sanciangco, and N. J. Tolentino, “Digitally enabled collaborative governance for sustaining bus reforms on the edsa busway in metro manila,” *Research in Transportation Economics*, vol. 113, p. 101633, 2025.
- [10] J. M. B. Espino, R. G. L. Larraquel, A. S. R. L. Purisima, A. M. B. Valenzuela, and M. N. Borromeo, “Impacts of different rainfall intensities on key traffic flow parameters at north luzon expressway using underwood’s exponential model,” in *Proceedings of the 25th Annual Conference of the Transportation Science Society of the Philippines*, 2018.

- [11] J. Mangaluz, “Flooding affects bus trips; train ops continue,” *The Philippine Star*, July 2024.
- [12] Philippine Information Agency, “Edsa bus carousel to undergo emergency road repair.” PIA News, Aug. 2023.
- [13] S. Locus, “Dpwh to close portions of edsa bus carousel for emergency repairs,” 2023.
- [14] A. Ceder, *Public Transit Planning and Operation: Theory, Modeling and Practice*. Oxford, UK: Butterworth-Heinemann, 1st ed., 2007.
- [15] Y. Wang, D. Zhang, L. Hu, Y. Yang, and L. H. Lee, “A data-driven and optimal bus scheduling model with time-dependent traffic and demand,” *IEEE Transactions on Intelligent Transportation Systems*, vol. 18, no. 9, pp. 2443–2452, 2017.
- [16] T. Chen, X. Chen, and Y. Wang, “Dynamic bus dispatching via deep reinforcement learning: A survey,” *arXiv preprint arXiv:2401.12455*, 2024.
- [17] J. Wang and L. Sun, “Dynamic holding control to avoid bus bunching: A multi-agent deep reinforcement learning framework,” *Transportation Research Part C: Emerging Technologies*, vol. 116, p. 102661, 2020.
- [18] C. F. Daganzo, “A headway-based approach to eliminate bus bunching: Systematic analysis and comparisons,” *Transportation Research Part B: Methodological*, vol. 43, no. 10, pp. 913–921, 2009.
- [19] Y. Huang *et al.*, “Bus arrival time prediction using machine learning techniques in uncertain environments,” *IEEE Access*, vol. 7, pp. 93224–93235, 2019.
- [20] S. Patil *et al.*, “Advanced predictive analytics for public transit control and fleet scheduling,” *IEEE Transactions on Intelligent Transportation Systems*, vol. 26, no. 2, pp. 11176–11186, 2025.
- [21] J. E. Barrera Hernandez, L. E. Tarazona Torres, A. Tabares, and D. Álvarez-Martínez, “Optimization of bus dispatching in public transportation through a heuristic approach based on passenger demand forecasting,” *Smart Cities*, vol. 8, no. 3, 2025.
- [22] A. Usman, T. M. Ali, and C. Irawan, “Machine learning approaches for real-time traffic density estimation and public transport optimization,” *Engineering Proceedings*, vol. 107, no. 1, 2025.
- [23] T. Alexandre, F. Bernardini, J. Viterbo, and C. E. Pantoja, “Machine learning applied to public transportation by bus: A systematic literature review,” *Transportation Research Record*, vol. 2677, no. 7, pp. 639–660, 2023.
- [24] J. Pang *et al.*, “A predictive holding control strategy for bus transit reliability using real-time information,” *IEEE Transactions on Intelligent Transportation Systems*, vol. 20, no. 10, pp. 3874–3884, 2019.
- [25] D. Verbich and A. El-Geneidy, “The impact of severe weather conditions and vehicle breakdowns on dynamic transit control reliability,” *Transportation Research Part A: Policy and Practice*, vol. 144, pp. 112–126, 2021.

- [26] A. Momenikorbekandi and M. Abbod, “Intelligent scheduling based on reinforcement learning approaches: Applying advanced q-learning and sarsa reinforcement learning models for the optimisation of job shop scheduling problems,” *Electronics*, vol. 12, no. 23, 2023.
- [27] J. Rodriguez, H. N. Koutsopoulos, S. Wang, and J. Zhao, “Cooperative bus holding and stop-skipping: A deep reinforcement learning framework,” *Transportation Research Part C: Emerging Technologies*, vol. 155, p. 104308, 2023.
- [28] J. Wang and L. Sun, “Robust dynamic bus control: A distributional multi-agent reinforcement learning approach,” *IEEE Transactions on Intelligent Transportation Systems*, vol. 24, no. 4, pp. 4075–4088, 2023.
- [29] Y. Zhao, D. A. Ramli, and M. C. Lim, “Dynamic bus holding control using spatial-temporal data: A deep reinforcement learning approach,” in *AI 2022: Advances in Artificial Intelligence, LNAI 13728*, pp. 586–598, Springer, 2022.
- [30] Y. Zhang and L. Zheng, “Single agent robust deep reinforcement learning for bus fleet control,” 2025.
- [31] J. Wang and L. Sun, “Multi-objective multi-agent deep reinforcement learning to reduce bus bunching for multilane services with a shared corridor,” *Transportation Research Part C: Emerging Technologies*, vol. 155, p. 104309, 2023.
- [32] F. Christianos, G. Papoudakis, A. Rahman, and S. V. Albrecht, “Scaling multi-agent reinforcement learning with selective parameter sharing,” in *Proceedings of the 38th International Conference on Machine Learning (ICML)*, vol. 139 of *Proceedings of Machine Learning Research*, pp. 1989–1998, PMLR, 2021.
- [33] C. Yu, A. Velu, E. Vinitzky, J. Gao, Y. Wang, A. Bayen, and Y. Wu, “The surprising effectiveness of PPO in cooperative multi-agent games,” in *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, vol. 35, 2022.
- [34] Z. Li, K. Cai, and P. Zhao, “Departure scheduling for multi-airport system using multi-agent reinforcement learning,” in *2023 IEEE/AIAA 42nd Digital Avionics Systems Conference (DASC)*, pp. 1–9, 2023.
- [35] W. Sun, Y. Zou, N. Guan, X. Zhang, G. Du, and Y. Wen, “Graph attention network-based deep reinforcement learning scheduling framework for in-vehicle time-sensitive networking,” *IEEE Transactions on Industrial Informatics*, vol. 20, no. 7, pp. 9825–9836, 2024.
- [36] T. Zuo, H. Liu, S. Yang, W. Wang, Y. Peng, and R. Wang, “A reinforcement learning method for automated guided vehicle dispatching and path planning considering charging and path conflicts at an automated container terminal,” *Journal of Marine Science and Engineering*, vol. 14, no. 1, 2026.
- [37] X. Huang, Y. Tian, J. Li, N. Zhang, X. Dong, Y. Lv, and Z. Li, “Joint autonomous decision-making of conflict resolution and aircraft scheduling based on triple-aspect improved multi-agent reinforcement learning,” *Expert Systems with Applications*, vol. 275, p. 127024, 2025.

- [38] R. Bokade, X. Jin, and C. Amato, “Multi-agent reinforcement learning based on representational communication for large-scale traffic signal control,” *IEEE Access*, vol. 11, pp. 47646–47658, 2023.
- [39] Y. Cao, Z. Xu, and M. Mei, “Train rescheduling method based on multi-agent reinforcement learning,” in *2022 IEEE 6th Advanced Information Technology, Electronic and Automation Control Conference (IAEAC)*, pp. 301–305, 2022.
- [40] F. Wang, P. Ding, Y. Bi, and J. Qiu, “A multi-agent deep reinforcement learning approach for multiple agvs scheduling in automated container terminals,” in *2024 China Automation Congress (CAC)*, pp. 3564–3569, 2024.
- [41] L. Nie, D. Qi, B. Liu, P. Li, H. Bao, and H. He, “Cmrm: Collaborative multi-agent reinforcement learning for multi-objective traffic signal control,” *IEEE Transactions on Consumer Electronics*, vol. 71, no. 2, pp. 2793–2805, 2025.
- [42] Q. Xu, Z. Zhang, J. Li, and X. Qi, “Hierarchical multi-agent reinforcement learning algorithm for multi-uav roundup strategy,” *Applied Mathematical Modelling*, vol. 155, p. 116728, 2026.
- [43] W. Chen, K. Zhou, and C. Chen, “Real-time bus holding control on a transit corridor based on multi-agent reinforcement learning,” in *Proceedings of the 19th IEEE International Conference on Intelligent Transportation Systems (ITSC)*, (Rio de Janeiro, Brazil), pp. 100–106, 2016.
- [44] K. Katzilieris, E. Kampitakis, and E. I. Vlahogianni, “A multi-agent reinforcement learning framework for integrated traffic signal control and dynamic bus lane access management,” *Transportation Research Part C: Emerging Technologies*, vol. 182, p. 105391, 2026.
- [45] D. Liu, F. Xiao, J. Luo, and F. Yang, “Deep reinforcement learning-based holding control for bus bunching under stochastic travel time and demand,” *Sustainability*, vol. 15, no. 14, p. 10947, 2023.
- [46] H. Shi, Q. Nie, S. Fu, X. Wang, Y. Zhou, and B. Ran, “A distributed deep reinforcement learning-based integrated dynamic bus control system in a connected environment,” *Computer-Aided Civil and Infrastructure Engineering*, vol. 37, no. 15, pp. 2016–2032, 2022.
- [47] Philippine Daily Inquirer, “DOTr stops EDSA busway consortium operations as Rolly hits Metro Manila.” Inquirer.net news report, Nov. 2020. Documents suspension of EDSA busway operations under Typhoon Signal No. 4.
- [48] P. Henderson, R. Islam, P. Bachman, J. Pineau, D. Precup, and D. Meger, “Deep reinforcement learning that matters,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, 2018.
- [49] R. Agarwal, M. Schwarzer, P. S. Castro, A. Courville, and M. G. Bellemare, “Deep reinforcement learning at the edge of the statistical precipice,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 34, 2021.
- [50] M. Patil, Q. Ahmed, and S. Midlam-Mohler, “Travel time and weather-aware traffic forecasting in a conformal graph neural network framework,” *IEEE Transactions on Intelligent Transportation Systems*, vol. 26, no. 12, pp. 21734–21744, 2025.

- [51] A. Che, Z. Wang, and C. Zhou, “Multi-agent deep reinforcement learning for recharging-considered vehicle scheduling problem in container terminals,” *IEEE Transactions on Intelligent Transportation Systems*, vol. 25, no. 11, pp. 16855–16868, 2024.
- [52] Y. Ju, Y. Chen, Z. Cao, L. Liu, Q. Pei, M. Xiao, K. Ota, M. Dong, and V. C. M. Leung, “Joint secure offloading and resource allocation for vehicular edge computing network: A multi-agent deep reinforcement learning approach,” *IEEE Transactions on Intelligent Transportation Systems*, vol. 24, no. 5, pp. 5555–5569, 2023.
- [53] Y. Yang, J. Cheng, and Y. Liu, “An overview of solutions to the bus bunching problem in urban bus systems,” *Frontiers of Engineering Management*, 2024.
- [54] L. Wasserman, *All of Statistics: A Concise Course in Statistical Inference*. New York: Springer, 2004.
- [55] Y. Tang, A. Qu, X. Jiang, B. Mo, S. Cao, J. Rodriguez, H. N. Koutsopoulos, C. Wu, and J. Zhao, “Robust reinforcement learning strategies with evolving curriculum for efficient bus operations in smart cities,” *Smart Cities*, vol. 7, no. 6, pp. 3658–3677, 2024.
- [56] P. A. Lopez, M. Behrisch, L. Bieker-Walz, J. Erdmann, Y.-P. Flötteröd, R. Hilbrich, L. Lücken, J. Rummel, P. Wagner, and E. Wießner, “Microscopic traffic simulation using SUMO,” in *2018 21st International Conference on Intelligent Transportation Systems (ITSC)*, pp. 2575–2582, IEEE, 2018.
- [57] J. K. Terry, B. Black, N. Grammel, M. Jayakumar, A. Hari, R. Sullivan, L. Santos, R. Perez, C. Horsch, C. Dieffendahl, N. L. Williams, Y. Lokesh, and P. Ravi, “PettingZoo: Gym for multi-agent reinforcement learning,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 34, 2021.
- [58] M. Towers, A. Kwiatkowski, J. K. Terry, J. U. Balis, G. De Cola, T. Deleu, M. Goulão, A. Kallinteris, M. Krimmel, A. KG, R. Perez-Vicente, A. Pierré, S. Schulhoff, J. J. Tai, H. Tan, and O. G. Younis, “Gymnasium: A standard interface for reinforcement learning environments,” *arXiv preprint arXiv:2407.17032*, 2024.
- [59] UK Highways Agency, “Design manual for roads and bridges, volume 12, section 2: Traffic appraisal in urban areas,” tech. rep., The Stationery Office, London, 1996.
- [60] Federal Highway Administration, “Traffic analysis toolbox volume iii: Guidelines for applying traffic microsimulation modeling software,” Tech. Rep. FHWA-HOP-18-036, U.S. Department of Transportation, 2019.
- [61] R. Bellman, “A Markovian decision process,” *Journal of Mathematics and Mechanics*, vol. 6, no. 5, pp. 679–684, 1957.
- [62] S. J. Bradtke and M. O. Duff, “Reinforcement learning methods for continuous-time Markov decision problems,” in *Advances in Neural Information Processing Systems*, vol. 7, pp. 393–400, 1995.

- [63] M. L. Littman, “Markov games as a framework for multi-agent reinforcement learning,” in *Proceedings of the 11th International Conference on Machine Learning (ICML)*, pp. 157–163, Morgan Kaufmann, 1994.
- [64] H. van Hasselt, A. Guez, and D. Silver, “Deep reinforcement learning with double Q-learning,” in *Proceedings of the Thirtieth AAAI Conference on Artificial Intelligence*, pp. 2094–2100, 2016.
- [65] V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, G. Ostrovski, *et al.*, “Human-level control through deep reinforcement learning,” *Nature*, vol. 518, no. 7540, pp. 529–533, 2015.
- [66] B. Efron and R. J. Tibshirani, *An Introduction to the Bootstrap*. New York: Chapman & Hall/CRC, 1993.